On Competitive Analysis for Polling Systems (Extended Version)

Jin Xu* Natarajan Gautam†

Abstract

Polling systems have been widely studied, however most of studies focus on renewal processes for arrivals and random variables for service times. However, there is a need driven by applications to study arbitrary arrivals (not restricted to time-varying or in batches), and service times revealed before start of service. To address that need, our work, different from frequently studied polling systems, considers a total completion time minimization problem and for the first time provides the worst case analysis for online scheduling policies on generic polling systems. We provide necessary and sufficient conditions for the existence of constant competitive ratio for polling systems on general settings, and also the competitive ratios for several well-studied policies such as cyclic exhaustive, gated and l-limited policies, Stochastic Largest Queue policy, One Machine policy and Gittins Index policy for polling systems. We show that any policy with purely static, or queue-length based or job-processing-time based routing discipline does not have a competitive ratio smaller than k, where k is the number of queues. Finally, a mixed strategy is provided for a practical scenario where setup time is large but bounded.

Keywords— Scheduling, Online Algorithm, Competitive Ratio, Parallel Queues with Setup Times

1 Introduction

This research has been motivated by operations in additive manufacturing machines. As an illustration, consider a 3D printing machine that uses a particular material, that we informally call 'ink', to print jobs that require that ink. Jobs of the same prototype are printed using the same ink, and when a different prototype (for simplicity, say a different color) is to be printed, a different ink is required and the machine undergoes a setup that takes time to switch inks. Thus we model this problem as a polling system where the server polls a queue, processes jobs, incurs a switch over time and processes another queue, and so on. In more formal terms, the 'ink' besides the material, could be processing temperature, equipment setting and other processing requirements that need setup between different job prototypes.

Another interesting feature of 3D printing is that it is possible to reveal workload of jobs (i.e., processing times) upon arrival. This is because before getting printed, the printing requirements such as temperature, nozzle route, printing speed and so on are specified for the job, and using that one can easily acquire the printing time before processing. Therefore, assuming the workload of a job is stochastic at the start of processing is unnecessary, although many other queueing research articles do so. In this paper, we assume that the workload of jobs could be arbitrary, and can be revealed deterministically upon arrival. Further, the 3D printer that prints customized parts usually receives jobs with different processing requirements.

*Email: jinxu@tamu.edu, Industrial and Systems Engineering, Texas A&M University
†Corresponding author, Email: gautam@tamu.edu, Industrial and Systems Engineering, Texas A&M University

Also, job arrivals could be time-varying non-renewal, batch arrivals, dependent or even arrivals without a pattern. It motivates us to consider the generic polling system with arbitrary arrivals, without imposing any stochastic assumptions, except for future arrivals.

Motivated by the 3D printing example, we consider a polling system with k ($k \geq 2$) queues served by one server, where a setup time occurs when the server switches from one queue to another. Such polling systems with arbitrary arrivals and revealed workload can be found in many application areas such as computer-communication systems, reconfigurable smart manufacturing systems and smart traffic systems. They have been widely studied and also recognized as both important and complex in the queueing literature [7]. However, the study of polling systems is far from complete. Most of works are from a stochastic standpoint, assuming certain distributions for arrival, service and switching processes. As a consequence, average performance is often discussed and analysis highly depends on these stochastic assumptions. There are very few articles on optimality and online scheduling algorithms for polling system without stochastic assumptions for jobs in the system [43]. With the advent of smart manufacturing and Internet of Things (IoT), it is possible to ascertain the processing times of jobs in the system prior to starting service. However, the arrival processes can be highly irregular and future arrivals can be completely uncertain and unpredictable. In our paper, we consider an online optimization framework and perform worst case analysis for generic polling systems that does not rely on any stochastic assumptions for arrival, service or switching processes, by assuming job processing times are revealed upon arrival and future arrival times and processing times are uncertain. More specifically, we consider a completion time minimization problem for such polling systems focusing on obtaining the worst case performance of certain online scheduling policies by benchmarking against the *deterministic optimal offline policy* on arbitrary job instances, which is also known as competitive ratio analysis. In this work we, for the first time, provide necessary and sufficient conditions for polling systems' online policies to have constant competitive ratios. Moreover, we derive competitive ratios for several widely used scheduling policies with known average performance, such as shortest remaining processing time first, serving the stochastic longest queue as well as cyclic exhaustive and gated policies. We suggest a mixed strategy in this paper that works under practical scenarios.

1.1 Problem Statement

We consider a single server system with k parallel queues. Jobs arrive at the system in a general pattern and then wait in one of the queues (based on their class) before getting served. The k queues correspond to the k classes of jobs (k is finite) in the system and each class of jobs has its own queue. Figure 1.1 shows a polling system with $k = 4$ queues. The processing time (workload) of the i^{th} arriving job p_i is revealed instantly upon its arrival time r_i. The server can serve jobs in the system in any order. However, a setup time τ is incurred when the server serves a job whose class is different from the previous one, i.e., switching from one queue to another. We assume that once the setup is in progress, it can be interrupted but it is not resumable. Next time the server still needs time τ to set up the queue. Information (r_j, p_j) about a future job j (for all j) remains unknown to the server until job j arrives in the system. The objective is to find service order for jobs and queues to minimize total completion time over all jobs, where the completion time of a job is the time period from time 0 to when the job gets served (exits the system). It is assumed that the number of jobs is an arbitrary large finite quantity. We wish to also state that research on polling system usually refer to queueing systems with stochastic arrivals and setup times, but in our paper we allow polling systems to have arbitrary arrival times and processing times, without stochastic assumptions.

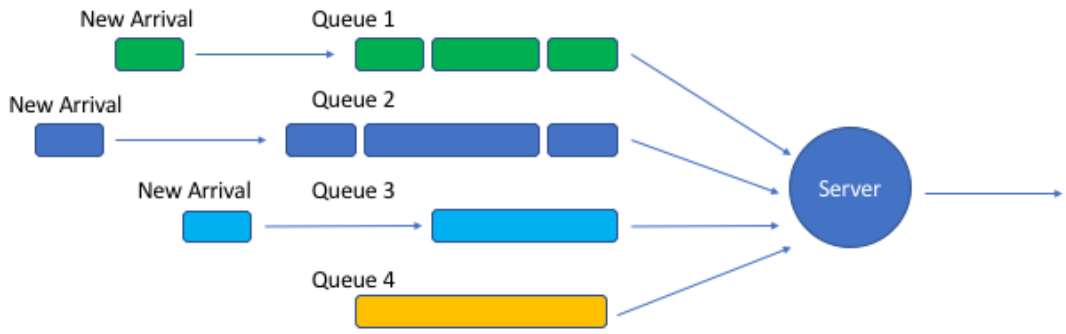

Figure 1.1: Polling System with Four Queues

1.2 Preliminaries

In this subsection we mainly introduce some concepts and terminologies that we use in this paper. Important notations in this paper are provided in Table 1.

Machine Scheduling Problems:

The polling system scheduling problem belongs to the class of one machine online scheduling problems since there is only one server (machine or scheduler) in the system. Using the notation for one machine scheduling problems [18], we write our problem as $1 \mid r_i, \tau \mid \sum C_i$, where 1 denotes there is one machine, r_i and τ mean the problem has release date and setup time constraints respectively, and $\sum C_i$ means the objective is to minimize the total completion time. This type of notations is widely used to briefly describe the input, constraints and objective for any scheduling problem. The notation is of the type $P_m \mid constraints \mid objective$. The first column in the notation denotes the number of machines in the system. For example, it could be 1 for one machine or P_m for m paralleled machines. The middle column denotes the constraints, in our paper we have r_i for releasing time constraints, and τ for setup time constraints, as well as other constraints that we would introduce (via $\tau \leq \theta p_{min}$ or $p_{max} \leq \gamma p_{min}$ to denote bound relations between setup time τ, upper bound workload $p_{max} = \sup_i p_i$ and lower bound workload $p_{min} = \inf_i p_i$; we call the workload variation is bounded if $p_{max} \leq \gamma p_{min}$ for constant γ). In other papers there could be *pmtn* for preemption allowance (the server can start working on another job with interrupting the current job and resuming it later), or *prec* for precedence constraints (one job must be processed before the other). If no constraint is specified in this column, it means all jobs are available at time 0 (or with a trivial arrival time 0), with no precedence constraints are imposed, preemption is not allowed and no setup times exist. In this paper, we assume jobs and setup times are non-resumable thus all the policies that we discuss are non-preemptive unless we specifically point out. The last column of the notation represents the objective for the scheduling problem (default to be minimization problems). In this paper it is $\sum C_i$, which means minimizing the total completion time. In other papers it could be C_{max} [24], which means minimizing the maximal completion time (makespan), or $\sum w_i C_i$ which is minimizing the weighted completion time [19], where w_i is the weight for job i.

Online and Offline Problems:

A job instance I with $n(I)$ number of jobs is defined to be a sequence of jobs with certain arrival times and processing times, i.e., $I = \{(r_i, p_i), 1 \leq i \leq n(I)\}$. In this work we mainly focus on an online scheduling problem, where online means r_i and p_i remain unknown to the server until job i arrives. In this paper we assume setup time τ is always revealed to be a constant for all queues. As a benchmark to evaluate the online problem, for a given job instance, the offline problem has entire information for $(r_1, r_2, ..., r_{n(I)})$ and $(p_1, p_2, ..., p_{n(I)})$, i.e., the future information is revealed to the server at time 0. The offline problem is usually of great complexity, for instance $1|r_i \mid \sum C_i$ and $1 \mid prec \mid \sum C_i$ are strongly NP-hard [19]. Hence, approximation or heuristic methods are needed. However it is important to note that the preemptive policy that serves the shortest remaining processing time (SRPT) is the optimal policy for $1|r_i, pmtn| \sum C_i$ with $\tau = 0$ (see [37]). SRPT is online and polynomial-time solvable, which can also be used as a benchmark for online scheduling policies to compare against.

Scheduling Policies:

A scheduling policy π decides the order that jobs get served and provides a feasible solution to the scheduling problem. In our paper we mainly focus on online policies, which means polices that adapt to online problems, without knowing the information for future jobs. It is also called non-anticipative in some literature [45, 6]. There are two types of scheduling policies in terms of pattern of decision making, one is called *deterministic,* and the other is called *randomized.* There is only one unique solution if a job instance is given to a deterministic policy. For example, SRPT is a deterministic policy. A randomized policy may toss a coin before making decisions, and the decisions may depend on the outcome of this coin toss [6]. This means for a random policy, its decision may depend on the outcome of its internal random variable, drawn from a certain distribution and this distribution is usually specified. Detailed discussions for randomized algorithms can be found in [38, 8]. If the same job instance is given to a randomized policy multiple times, each time the outcome may be different from the others due to the internal random variable having different outcomes.

Competitive Ratios:

Competitive ratio is a ratio between the solution obtained by an online algorithm and the *benchmark*. In this paper, the optimal solution to the offline problem is the benchmark that we want to compare our online algorithms against. Thus we define the competitive ratio ρ as $\sup_I \frac{C^\pi(I)}{C^*(I)} \leq \rho$ for any job instance I, where $C^\pi(I)$ is the completion time for job instance I by a deterministic scheduling policy π and $C^*(I)$ is the optimal completion time of the offline problem. We say a competitive ratio is tight if there exists an instance such that $\frac{C^\pi(I)}{C^*(I)} = \rho$. For a randomized policy π, the competitive ratio is defined as $\sup_I \frac{\mathbf{E}[C^\pi(I)]}{C^*(I)} \leq \rho$, where the expectation is taken over the internal random variable of the randomized policy. In certain circumstances, randomized policies have substantially lower competitive ratio, as shown in [8, 6].

1.3 Related Works

The one machine scheduling problems with minimizing the total completion time and considering setup times or costs have been widely studied from various perspectives. A detailed review of the literature can be found in [3, 2]. However, almost all of them are focused on solving the offline version of the problem where

Notations	Meaning	Notations	Meaning
r_i	Release date, the time when job i arrives in the system	p_i	Workload (processing time) for job i
p_{max}	Maximum workload, $p_{max} = \max_i\{p_i\}$	p_{min}	Minimum workload, $p_{min} = \min_i\{p_i\}$
τ	Setup time is τ for all queues	$I = \{r_i, p_i\}$, for $1 \leq i \leq n(I)$	Job instance, set of jobs with information about release date and processing time for all jobs in it
$C^\pi(I)$	Total completion time for jobs in instance I under policy π	$C^*(I)$	Total completion time for jobs in instance I under the optimal policy of the offline problem
C_i^π	The completion time for job i under policy π	$n(I)$	Number of jobs in job instance I
γ	Workload variation, a constant such that $p_{max} \leq \gamma p_{min}$	θ	A constant such that $\tau \leq \theta p_{min}$
k	Number of queues in the system	$\kappa = \max\{\frac{3}{2}\gamma, k+1\}$	Competitive ratio for cyclic-based and exhaustive-like policies, a constant
$\bar{I}$	Workload augmented instance of I, with $\bar{p}_i = p_{max}$ but the arrivals the same as arrivals in I	$\underline{I}$	Workload reduced instance of I, with $\underline{p_i} = p_{min}$ but the arrivals the same as arrivals in I
$\tilde{I}$	Setup time augmented instance of I, with $\tilde{p}_i = p_i + \tau$ but arrivals the same as arrivals in I	$\underset{\sim}{I}$	Setup time reduced instance of I, with $\underset{\sim}{p_i} = p_i + \tau$ but arrivals the same as arrivals in I
$\hat{I}$	Workload and setup time augmented instance of I, with $\hat{p}_i = p_{max} + \tau$ but arrivals the same as arrivals in I		

Table 1: Notations

Problem	Deterministic		Randomized	
	Lower Bounds	Upper Bounds	Lower Bounds	Upper Bounds
$1\|r_i, pmtn\|\sum C_i$	1	1 [37]	1	1 [37]
$1\|r_i, pmtn\|\sum w_i C_i$	1.073 [12]	1.566 [36]	1.038 [12]	$\frac{4}{3}$ [35]
$1\|r_i\|\sum C_i$	2 [20]	2 [20, 26, 32]	$\frac{e}{e-1}$ [38]	$\frac{e}{e-1}$ [8]
$1\|r_i\|\sum w_i C_i$	2 [20]	2 [4]	$\frac{e}{e-1}$ [38]	1.686 [17]
$1\|r_i, \frac{p_{max}}{p_{min}} \leq \gamma\|\sum w_i C_i$	$1+\frac{\sqrt{4\gamma^2+1}-1}{2\gamma}$ [41]	$1+\frac{\sqrt{4\gamma^2+1}-1}{2\gamma}$ [41]	Unknown	Unknown
$1\|r_i, \frac{p_{max}}{p_{min}} \leq \gamma\|\sum C_i$	Numerical [40]	$1+\frac{\gamma-1}{1+\sqrt{1+\gamma(\gamma-1)}}$ [40]	Unknown	Unknown

Table 2: Competitive Ratios for Single Machine Scheduling Problem without Setup Times (i.e., $\tau = 0$)

jobs may arrive over time, but all the information including release time and processing time is revealed at time 0 such as [31, 30]. This problem is strongly NP-hard since it is proved that the one machine scheduling problem without batch setup time but with release date, is strongly NP-hard [23, 19, 21]. Including the setup time further adds to the complexity of the problem.

In this paper we are more interested in the online problem where the current state of the system is known and future is uncertain. Thus our work also falls into the class of online single machine scheduling problems. Numerous research papers have shed light on the online single machine scheduling problem, without considering setup times. Table 2 summarizes the current state of art of competitive ratio analysis over existing online algorithms for single machine scheduling problem without setup times. Besides all the algorithms provided in Table 2, a recent work [27] provides a new method to approximate the competitive ratio for general online algorithms. However, these works mainly focus on solving problem without setup times, and they are not directly applicable to polling systems.

Only few articles so far have focused on the online single machine scheduling problem with queue setup times, i.e., the polling system, from a worst case perspective and considering optimality as well as approximation algorithms. The online makespan minimization problem for the polling system is considered in [10] and an $\boldsymbol{O}(1)$ algorithm is proved to exist, however the competitive ratio provided is very large. A 3-competitive online algorithm for the polling system is considered in [47], but only with identical jobs and number of queues $k = 2$. To the best of our knowledge, the online algorithms for general polling systems with setup time and worst case consideration have not been well studied.

However, there are articles that study polling systems from a stochastic perspective by assuming job arrivals, service times and setup times are stochastic. Average performance of policies is considered for different types of assumptions about randomness [43]. Exact mean value analysis for cyclic routing policies with exhaustive, gated and limited service disciplines for polling system with $M/G/1$ type queues have been provided in different ways by [13, 39, 34, 46, 14]. Service disciplines within queues (such as FCFS, Shortest Remaining Processing Time First and others) are discussed in [44], however the routing discipline (choose which queue to serve) and optimal service discipline between queues are not discussed. Optimal service policy for symmetric polling system is provided by [25], where symmetric means that jobs arrive evenly into each queue and jobs are stochastic and identical. Most of the works in this stochastic category are restricted to average performance analysis. The structure of the optimal solution to the general polling system yet remains unknown [7, 43]. Approximating algorithms for the polling system are very few, and none of those popular policies with proven average performance have the adaptivity in general settings. In this paper we provide the performance analysis for those policies without stochastic assumptions, and evaluate policies by

the worst case performance.

In summary, to the best of our knowledge, there is no work which provides optimal or approximating scheduling policy for polling systems with a general setting and minimization of the total completion times. In addition, the conditions under which polling system allows constant competitive ratios for online algorithms are also unknown. Besides, the competitive ratios for widely-used policies for polling systems (such as cyclic policies with exhausted and gated service discipline) remain unknown. The contribution of this paper is threefold: (1) Our work for the first time analyzes polling systems without stochastic assumptions and evaluates policy performance by competitive ratio, provides the necessary and sufficient conditions for polling systems to have constant competitive ratio, and proves competitive ratios for some well-studied policies such as cyclic exhaustive and gated policy. (2) We provide a lower bound for the competitive ratio for all the possible online policies, and show that no online algorithm can have competitive ratio smaller than this lower bound. (3) We also provide online policies that balance future uncertainty and utilize known information, which may open up a new research direction that would benefit from revealing information and reducing variability. This paper is organized as follows: in Section 2 we consider the case of bounded workload variation and obtain competitive ratios for various policies; in Section 3 we consider the case with unbounded workload variation; in Section 4 we provide a mixed strategy to deal with a practical scenario of large but bounded setup times; we make concluding remarks in Section 5.

2 Polling System with Bounded Workload Variation

From [37, 16] we know that in the single machine single queue case with minimizing completion times but no setup times, it is always beneficial to schedule jobs preemptively with the smallest remaining processing time (SRPT) first. The reason is that by doing this, small jobs are quickly processed thus number of waiting jobs is reduced. The idea of scheduling small workload first can be used in polling systems as well. An efficient scheduling policy may need to avoid the case where a large number of jobs are waiting in queue while a single large job is in process, and also the case when a large number of small jobs waiting in a queue, but the server is serving other queues and not able to switch to this queue immediately. Thereby, the main idea of designing a scheduling policy is to avoid either of the extreme cases happening. A good online policy should balance job priority and queue priority so that small jobs are processed in a timely manner and switching does not happen frequently. The priority of a job is usually determined by the job information, such as processing time and release date. In contrast, queue priority is usually determined by queue information such as number of waiting jobs, sum of remaining workload in a queue and so on. If the variation of processing time is large and setup times are trivial, then perhaps the server should favor job priority more than queue priority since a large job in process may cause a much larger delay than switching. In the case where variation of processing times is small, the server may need to favor queue priority.

We leave the discussion for unbounded variation to Section 3 and in this section we mainly discuss the case where workload variation is bounded (i.e., $p_{max} \leq \gamma p_{min}$ for some constant γ, recall that $p_{max} = \sup_i p_i$ and $p_{min} = \inf_i p_i$). The case of bounded workload variation, in the 3D printing example that we introduced earlier in this paper, corresponds to the scenario in which jobs have similar printing requirements but different colors, i.e., have needing to switch but have similar processing times. Notice that once the variation of processing times is small, queue priority should be favored. It is important for a scheduling policy to decide when to switch out from the current queue, how many jobs to serve in a certain queue and in which order do these jobs get served. We call it *service discipline*. It is also important to decide which queue to

switch into when switching occurs and we call the way of selecting queue the *routing discipline*. We shall show later that when workload variation is small, service discipline does not impact the policy performance greatly, as long as it follows an exhaustive-like structure. Thereby in this section, we mainly focus our discussion on the routing discipline. There are many widely used routing disciplines and here we classify them into two types. One is called *static* routing discipline, which means the server always follows a fixed routing order (also called a general routing table), such as the policy provided in [5]. The other is called *dynamic* routing discipline, which means the server follows a dynamic routing policy which may use any available information as the processing goes on, such as Stochastic Largest Queue (SLQ) provided in [25], or selecting queues randomly such as random routing provided in [22]. For static routing disciplines, we focus our discussion on the cyclic routing policy (also called periodic), and later in this section we shall show that cyclic routing is the optimal static routing polices in terms of worst case performance, since it treats each queue equally and each queue is visited at the same frequency. We will also show that random routing discipline is generally no better than static ones in terms of the worst case performance, which may be surprising and non-intuitive at beginning, but it indeed reflects the underlying difference between average performance with worse case performance. Since in this section we only consider the case where workload variation is bounded, we assume the maximal processing time (across all queues) for an arbitrary job is bounded by a constant ratio of minimal possible processing time across all queues, i.e., $p_{max} \leq \gamma p_{min}$ for some constant γ. And we assume the setup time for all queues is fixed as τ. So our problem is denoted as $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$ where $p_{max} \leq \gamma p_{min}$ is a constraint imposed. Unlike in [40] where p_{min} is assumed to be non-zero, here we also allow $p_{min} = p_{max} = 0$, for which we assume $\gamma = 1$.

2.1 Cyclic Based and Exhaustive-like Policies

In this subsection we mainly focus on policies which use cyclic routing discipline, and serve each queue in an exhaustive manner. A queue is exhausted means there is no job with unfinished workload in the queue by the time the server leaves it. These policies form a policy set Π_r, which we will define later. We begin our discussion with a set of policies Π_1 called Cyclic Exhaustive Policies with Skipping Empty Queues, as shown in Algorithm 1. This set contains all the policies that serves queues in a cyclic way (from queue 1 to k and then again from 1 to k and so on), serving each queue exhaustively and waiting in the queue for a certain amount of time before switching out. Once the server decides to switch out, it selects the next non-empty queue to switch to. If all queues are empty, the server will idle at the last queue which it served. Note that for an arbitrary $\pi \in \Pi_1$, during each visit to queue i, the server has to serve all the available jobs in queue i before switching out. After serving all the jobs in queue i, the server is allowed to wait in queue i for some extra time to receive more arrivals. If the server processes n_i^w jobs in its w^{th} visit to queue i, it can stay in queue i for at most $n_i^w p_{max}$ amount of time in this visit. If a new arrival occurs at queue i during the time that the server is waiting, then $n_i^w \leftarrow n_i^w + 1$ and the server can process this job at any time before it switches out, as long as the server does not stay in the queue for time longer than the updated $n_i^w p_{max}$. This actually allows for a wide class of service disciplines. Note that if during w^{th} visit to queue i, all the jobs have workload p_{max}, then the server will switch out when a queue is exhausted. Also note that we do not specify the service order of jobs for policies in Π_1. As long as a policy satisfies the description of Algorithm 1, it belongs to Π_1. The next theorem provides the competitive ratio for policies in Π_1. Since the proofs of this and some subsequent theorems are lengthy, they are provided in the appendices.

Theorem 1. *Any policy in* Π_1 *has competitive ratio of* $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$ *for the polling system* $1 \mid$

$r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$. *This competitive ratio is tight when* $\frac{3}{2}\gamma \leq k+1$.

Proof. The main idea of the proof is to consider each 'batch', which are the jobs that are served by the online policy during each visit to a queue. We consider the first arrival time and service start time of each batch, and use inductive argument to compare them with the optimal solution. The detail of the proof is in Appendix A.

□

Algorithm 1 Cyclic Exhaustive Policies with Skipping Empty Queues Π_1

Require: Instance I
$server = 1; w = 1$
while I has not been fully processed **do**
    Process all jobs that are present in $queue_{server}$ during server's w^{th} visit to $queue_{server}$ (the total number of jobs is denoted as n^w_{server}), where the length of each visit period is at most $n^w_{server} p_{max}$
    if queue $server < j \leq k$ is not empty **then**
        $server = server + min\{j\}$
    else if queue $1 \leq j \leq server - 1$ is not empty **then**
        $server = min\{j\}$; $w = w + 1$
    else
        $server = l$ if next arrival occurs at queue l; $w = 1$
    end if
end while
return Total completion time $C^{\pi_1}(I)$

Notice that if $p_i = 1$ for all jobs i, then $\gamma = 1$ and $\kappa = 3$ if $k = 2$, which is the result shown in [47]. However Theorem 1 implies a stronger result which allows multiple queues ($k \geq 2$) and p_i's to be different. Next we show a set of policies Π_2 which keep switching even when the system is empty, called Cyclic Exhaustive Policies without Skipping Empty Queues and shown as Algorithm 2. If a policy serves queues in a cyclic manner, skips empty queues and switches out immediately when a queue is exhausted, then it belongs to Π_2. This policy is well-studied and also has provable average performance for $M/G/1$ type queues [13, 39, 34, 46]. Here we show the competitive ratio for the policies in Π_2 is also κ.

Algorithm 2 Cyclic Exhaustive policy without Skipping Empty Queues Π_2

Require: Instance I
$server = 1; w = 1$
while I has not been fully processed **do**
    Process all jobs that are present in $queue_{server}$ during server's w^{th} visit to $queue_{server}$ (the total number of jobs is denoted as n^w_{server}), where the length of each visit period is at most $n^w_{server} p_{max}$
    $server = Rem(server, k) + 1$
    if $server == 1$ **then**
        $w = w + 1$
    end if
end while
return Total completion time $C^{\pi_2}(I)$

Theorem 2. *Any policy in* Π_2 *has competitive ratio of* $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$ *for the polling system* $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$. *This competitive ratio is tight when* $\frac{3}{2}\gamma \leq k+1$.

Proof. The proof of Theorem 2 is similar to the one for Theorem 1. A detailed proof is provided in Appendix B. □

Notice that the policies in Π_1 and Π_2 are different only in the way that the server deals with empty queues, and all the policies in Π_1 and Π_2 use exhaustive service discipline. Besides the exhaustive discipline, the gated discipline is often discussed because its average performance for $M/G/1$ type queues is also provable under cyclic routing policy without skipping empty queues [39, 46]. Under gated discipline, the server only serves the jobs that are in the queue before the server sets up the queue, and jobs that arrive after that time would be left to the next cycle of service. We do not specify the service order for the gated discipline. We let Π_3 be the set of polices that follow cyclic routing discipline without skipping the empty queues and serve each queue with gated discipline. We also allow server to wait after clearing a queue under Π_3. Once the server has set up a queue, the number of jobs that are served during this visit is known, however during waiting there may be arrivals to other queues. Thus waiting may still be beneficial for some specific job instances. The policies which do not skip empty queues are provided as Algorithm 3. Similarly, we can have a policy set Π_4 in which policies are based on cyclic discipline and gated service discipline allowing waiting, but skipping empty queues when switching. We do not state the algorithm Π_4 in detail here, but policies in Π_4 also have competitive ratio κ, as shown in the following theorem.

Algorithm 3 Gated Cyclic Policies without Skipping Empty Queues Π_3

Require: Instance I
1: $server = 1; w = 1$
2: **while** I has not been fully processed **do**
3: Process all jobs that are present in $queue_{server}$ upon server's setup for $queue_{server}$ (the total number of jobs is denoted as n^w_{server}), where the length of each visit period is at most $n^w_{server} p_{max}$
4: $server = Rem(server, k) + 1$
5: **if** $server == 1$ **then**
6: $w = w + 1$
7: **end if**
8: **end while**
9: **return** Total completion time $C^{\pi_3}(I)$

Theorem 3. *Any policy in* Π_3 *and* Π_4 *has competitive ratio of* $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$ *for the polling system* $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$. *This competitive ratio is tight when* $\frac{3}{2}\gamma \leq k+1$.

Proof. The proof of Theorem 3 is also similar to the one for Theorem 1. A detailed proof is provided Appendix C. □

So far we have shown the competitive ratios for policies based on exhaustive and gated disciplines, with and without skipping empty queues. Notice that all the policies in $\Pi_i, i = 1, ..., 4$ use cyclic as the routing discipline and have the same competitive ratio. We let these policies form a policy set Π_r, i.e., $\Pi_r = \cup_{i=1}^4 \Pi_i$. Again, we do not specify in Π_r the service order for jobs during the server's visit to a queue. It could be First Come First Served (FCFS), Shortest Processing Time First (SPT) or any other non-preemptive processing order, but all of them result in the same competitive ratio. It is also interesting to note that different service disciplines in Π_r result in the same competitive ratio when $\frac{3}{2}\gamma \leq k+1$ due to the tightness of competitive ratio is this case, although revealing processing times may bring more job information for scheduling. It is not a coincidence that all policies in Π_r have the same competitive ratio. All the policies in Π_r follows an

exhaustive-like approach, even for the gated discipline. Gated policies in Π_3 for $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$ exhaust all the jobs that arrive before the queue is set up, and if a large number of jobs arrive after the queue is set up, they will later be processed in the next round. In fact, Theorem 4 presented next shows exhaustive is the optimal service discipline in the case where $p_{max} = p_{min}$.

Theorem 4. *For the polling system $1|r_i, \tau, p_{min} = p_{max}|\sum C_i$, there always exists an exhaustive policy which outperforms a non-exhaustive policy in terms of total completion time.*

Proof. The case where $p_i = 0$ is trivial. Now we let $p_i = 1$ with appropriate units. Since preemption is not allowed and all the jobs are identical, the server only needs to decide when to switch and which queue to switch to. If there are jobs in the queue where the server is, there are two options for the server: to continue serving the next job in this queue, or to switch to a nonempty queue and later come back to this queue again. Suppose at time 0 the server is at queue 1 and there is an unfinished job in queue 1, then the server has to come back after it switches out. Suppose under a non-exhaustive policy $\pi^{'}$, the server chooses to switch to some queue(s) and come back to queue 1 at time T. Suppose the server served instance $I^{'}$ during this period. Suppose there is an adversary policy which has the same instance at time 0, and this policy chooses to serve one more job in queue 1, and then follows all decisions that policy $\pi^{'}$ has made (including waiting). Note that every decision policy $\pi^{'}$ made is available to the adversary policy, since the adversary policy serves one more job before leaving queue 1. The total completion time under $\pi^{'}$ is $C^{\pi^{'}} = 1 + T + C(I^{'})$, and the completion time achieved by the adversary is $C^{ad} = 1 + C(I^{'}) + n(I^{'})$, where $C(I^{'})$ is the completion time of instance I served by policy $\pi^{'}$ during $(0, T]$. Thus $C^{ad} - C^{\pi^{'}} \leq n(I^{'}) - T \leq 0$. Notice that the makespan of these two schemes are the same (including the final setup time of queue 1 for the adversary). The theorem is proved. □

Recall that all the policies in Π_r use the cyclic routing discipline, which is a static routing discipline. Next we show in Theorem 5, the cyclic routing discipline results in the smallest competitive ratio among all the static disciplines.

Theorem 5. *No online policy with static or random routing discipline can guarantee a competitive ratio smaller than k for $1|r_i, \tau, p_{max} \leq \gamma p_{min}|\sum C_i$. Further, cyclic is the optimal static routing discipline for such a system. No online policy has a competitive ratio smaller than 2 for $1|r_i, \tau, p_{max} \leq \gamma p_{min}|\sum C_i$.*

Proof. Since $1|r_i, \tau, p_i = 1|\sum C_i$ is a special case of $1|r_i, \tau, p_{max} \leq \gamma p_{min}|\sum C_i$, we only need to show the result holds for $1|r_i, \tau, p_i = 1|\sum C_i$. We first show the case for static routing policies. For an arbitrary policy that follows a static routing discipline, we suppose the server starts from queue 1, and queue k is the last one visited. Before the server visits queue k, queue i $(1 \leq i \leq k-1)$ has been visited v_i times (suppose $v_{(1)} \leq v_{(2)} \leq ... \leq v_{(k-1)}$ is the ascending order for v_i's). We construct a special job instance I by assuming there is one job arriving at each queue i every time the server visits queue i for $i = 1, ..., k-1$. Also we suppose there are n_k jobs at queue k at time 0 for a large n_k. Then we have

$$C^{\pi}(I) \quad \geq \quad n_k\tau(\sum_{i=1}^{k-1} v_i + 1) + g(I),$$

$$C^{*}(I) \quad = \quad (kv_{(1)} + ... + 2v_{(k-1)} + n_k)\tau + g(I),$$

and $\frac{C^\pi(I)}{C^*(I)} \geq \sum_{i=1}^{k-1} v_i + 1$ if we let $\tau = (n_k)^2$ and $n_k \to \infty$. To achieve the smallest ratio we need $v_i = 1$ for $i = 1, ..., k-1$. Thus cyclic is the optimal static routing discipline. Notice a random routing policy always generates a routing table randomly. Thus the result holds for random routing policies as well.

The lower bound competitive ratio for $1|r_i, \tau, p_i = 1|\sum C_i$ is 2 as given in [47]. Since problem $1|r_i, \tau, p_i = 1|\sum C_i$ is a special case for $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$ and $1|r_i, \tau|\sum C_i$, we know no online algorithm can have smaller competitive ratio than 2 for $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$ and $1|r_i, \tau|\sum C_i$. □

So far we have shown that for problem $1|r_i, \tau, p_{max} \leq \gamma p_{min}|\sum C_i$, the policies in Π_r all have competitive ratio $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$. From Theorem 5 we know the smallest competitive ratio based on cyclic routing discipline is at least k. The problem whether there is either a lower bound competitive ratio greater than k, or an online policy whose competitive ratio is smaller than κ remains open. When $\gamma \to \infty$, then $\kappa \to \infty$. This means the competitive ratio we prove goes to infinity, but it does not mean that policies in Π_r all have infinite competitive ratios, since the competitive ratio is tight only for $\frac{3}{2}\gamma \leq k+1$ as shown in Theorems 1, 2 and 3. However, Theorem 6 described next states that policies in Π_r do not have constant competitive ratio if γ is infinite.

Theorem 6. *Polices in Π_r do not guarantee constant competitive ratios for $1|r_i, \tau|\sum C_i$.*

Proof. We prove the theorem by giving a special job instance I. We assume $p_{min} = 0$ and $p_{max} = p$ so that $\gamma = \infty$. Suppose at time 0 each of queue $i = 2, ..., k$ has one job of processing time p and queue 1 has no job. At time $\tau + \epsilon$ there is are n jobs arriving at queue 1, with each of these n jobs has processing time 0. For any policy π in Π_r, the server would either setup queue 1 at time 0 then switch to queue 2 at time τ, or setup queue 2 at time 0. In either of the case the server will be back to queue 1 when queue k is served in the first cycle. Then we have

$$C^\pi(I) \quad \geq \quad \frac{k(k-1)}{2}p + n(k-1)p + \tau\left((n+k-1) + (n+k-2) + ... + n\right),$$

and

$$C^*(I) \quad = \quad \frac{k(k-1)}{2}p + \tau\left((n+k-1) + (k-1) + ... + 1\right) + \epsilon(n+k-1).$$

Letting $p > (n)^2$ and $n \to \infty$, we have $\frac{C^\pi(I)}{C^*(I)} \to \infty$. □

2.2 Other Queue-length Based Policies

Note that not all policies with cyclic routing discipline have competitive ratio κ. The reason that policies in Π_r have constant competitive ratio is they all serve as many available jobs as possible in a queue, thus reduce the frequency of switching. Nonetheless, it is interesting to consider the performance of other popular policies. We first consider a policy called *l-limited* policy. This policy is also based on the cyclic routing discipline, however the server only serves at most l jobs during each visit to a queue, then switches to the next queue. If the next queue is empty, the server would nevertheless set up the queue. A detailed description for this policy and average performance for $M/G/1$ type queues can be found in [39, 14]. Interestingly, as we shall show in Corollary 7, no constant competitive ratio is guaranteed by this policy.

Corollary 7. *The l-limited policy ($l < \infty$, with or without skipping empty queues) does not have a constant competitive ratio for* $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$.

Proof. We prove this result by giving a special instance I. Suppose there are $(l*n)$ number of jobs $(l, n \in \mathbf{Z}_+)$ at every queue at time 0. The server firstly sets up queue 1 and serves l number of jobs then makes a tour with setting up $(k-1)$ queues and serving l jobs at each queue. After returning queue 1, the server will again set up queue 1 and serve l jobs. This process will be repeated for n times before the entire instance I is processed. Let $C^l(I)$ be the total completion time for the *l-limited* policy, we have

$$\frac{C^l(I)}{C^*(I)} = \frac{\frac{knl(knl+1)}{2} + \tau l\left(kn + (kn-1) + ... + 1\right)}{\frac{knl(knl+1)}{2} + \tau nl\left(k + (k-1) + ... + 1\right)}.$$

If we let $\tau > (n)^2$ and $n \to \infty$, then $\frac{C^l(I)}{C^*(I)} \to \infty$. □

Although from Theorem 6 we show policies in Π_r do not have constant competitive ratios for $1 \mid r_i, \tau \mid \sum C_i$, they have constant competitive ratios for $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$. Corollary 7 shows *l-limit* policy does not belong to Π_r, since it does not have a constant competitive ratio for either $1 \mid r_i, \tau \mid \sum C_i$ or $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$.

As provided in [25], to serve the *stochastic largest queue (SLQ)* is the optimal policy when the system is stochastically symmetric. It should be clear that here 'symmetric' means if an arrival occurs, it will join one of the k queues equally likely, and service time for jobs in different queues are identically distributed. It does not mean arrivals to each queue are following the same sample path or service time of different jobs are revealed to be identical. The following proposition shows that SLQ is the optimal policy for a symmetric and stochastic polling system.

Proposition 8. *For a symmetric polling system, the optimal policy is given by SLQ which is as follows: 1) The server serves jobs in a queue non-preemptively; 2)The server should neither idle nor switch when it is at a non-empty queue (exhaustive); 3)The server stays idling in the last queue it visits when the system is empty; 4) When a queue is finished, the server switches to the next queue with the largest number of jobs in queue.*

Proof. See [25]. □

For a symmetric stochastic polling system, although we know the arrival and service processes between queues are identical, once the arrival and service times (random variables) are revealed, they may not be the same. The following theorem says SLQ for the symmetric system is no longer optimal when information is revealed deterministically. Further, we show that SLQ does not have a constant competitive ratio in Corollary 9.

Corollary 9. *The SLQ does not have a constant competitive ratio for* $1|r_i, \tau, | \sum C_i$*, thus it is not the optimal policy for* $1|r_i, \tau| \sum C_i$.

Proof. We prove the result by giving a special instance I. Suppose there are only 2 queues in the system. The first queue has 1 job with processing time p at time 0, while the second queue has no job at time 0 but has n jobs arriving at time τ. SLQ will serve from queue 1 to queue 2, so that

$$C^{SLQ}(I) \quad = \quad \tau + 2n\tau + p + np,$$

and

$$C^{*}(I) \quad = \quad n\tau + 2\tau + p.$$

Let $n \to \infty$ and $p \to \infty$ we have $\frac{C^{SLQ}(I)}{C^{*}(I)} \to \infty$. □

2.3 Simulation-based Policies

Now we consider policies that are based on simulation results. There are many simulation-based online algorithms in the literature such as the *One Machine* policy in [32] and the *α-scheduling* provided in [8]. These policies make decisions based on the result of simulations on a virtual system. For instance, One Machine policy simulates a virtual system that has the same arrivals as the real system, however in the virtual system preemption is allowed and SRPT is the optimal policy. So One Machine policy schedules jobs in the order that they are scheduled in the virtual system. Simulation-based policies usually need to simulate a virtual online benchmark in parallel and use the simulation result. For the polling system, we may want to simulate policies on virtual instances. Here we introduce two instances $\underline{I}$ and $\bar{I}$ which we call workload reduced and augmented instance respectively, such that $\underline{I}$ and $\bar{I}$ are of the same arrivals as I but $\underline{I}$ is of processing time p_{min} and $\bar{I}$ is of processing time p_{max} for all jobs. We can construct policies for I by simulating the policies on $\underline{I}$ or $\bar{I}$ and following the simulation results. Now we show that a policy which follows the decision that $\pi \in \Pi_r$ makes on either $\underline{I}$ or $\bar{I}$ has competitive ratio $\gamma(k+1)$.

Corollary 10. *A policy π that works on I but follows the decision that a policy $\bar{\pi} \in \Pi_r$ makes on either $\underline{I}$ or $\bar{I}$ is of competitive ratio $\gamma(k+1)$ for $1|r_i, p_{max} \leq \gamma p_{min}, \tau|\sum C_i$.*

Proof. Since there is only one type of jobs in either $\underline{I}$ or $\bar{I}$, preemption is not needed. We suppose $\bar{\pi} \in \Pi_r$ is a policy that works on $\underline{I}$, then $C^{\bar{\pi}}(\underline{I}) \leq (k+1)C^{*}(\underline{I})$ where $C^{*}(\underline{I})$ is the optimal completion time for instance $\underline{I}$. Because $\bar{\pi}$ is not preemptive, when $\bar{\pi}$ starts processing a job in $\underline{I}$ at time t, we know the process will be done at time $t + p_{min}$. Thus we construct the policy π by letting π follow the same service order that $\bar{\pi}$ has. Easily we have $C^{\pi}(I) \leq \gamma C^{\bar{\pi}}(\underline{I})$ because $p_i \leq \gamma p_{min}$ for every job i in instance I. Since $\underline{I}$ is a reduced instance, we have $C^{*}(\underline{I}) \leq C^{*}(I)$. Thus $C^{\pi}(I) \leq \gamma C^{\bar{\pi}}(\underline{I}) \leq \gamma(k+1)C^{*}(\underline{I}) \leq \gamma(k+1)C^{*}(I)$.

If $\bar{\pi}$ works on $\bar{I}$, where instance $\bar{I}$ is the augmented instance for instance I, with workload for all jobs in $\bar{I}$ is p_{max}. Then, we have $C^{\pi}(I) \leq C^{\bar{\pi}}(\bar{I}) \leq (k+1)C^{*}(\bar{I})$. Let $C^{\bar{*}}(\bar{I})$ be the completion time for a new policy which works on $\bar{I}$ but always makes the same decisions as the optimal policy for I, then $C^{\bar{*}}(\bar{I}) \leq \gamma C^{*}(I)$. Using $C^{*}(\bar{I}) \leq C^{\bar{*}}(\bar{I})$ we have

$$C^{\pi}(I) \quad \leq \quad C^{\bar{\pi}}(\bar{I}) \leq (k+1)C^{*}(\bar{I}) \leq (k+1)C^{\bar{*}}(\bar{I}) \leq \gamma(k+1)C^{*}(I).$$

□

Corollary 10 says we can simulate policies from Π_r on both workload augmented and reduced instances and follow the decisions that simulated policies make. Interestingly we find that choosing either $\underline{I}$ or $\bar{I}$ to simulate

policies on results in the same competitive ratios. However, the competitive ratio for this simulation-based policy is larger than κ defined for policies in Π_r, since $\gamma(k+1)-(k+1) \geq 0$ and $\gamma(k+1)-\frac{3}{2}\gamma > 0$.

We conclude this section by noting that all of the policies we have discussed are mainly focused on queue priority, where policies in Π_r use cyclic routing discipline, and SLQ are purely queue-length based. These policies may have constant competitive ratios under the bounded condition for the workload. In the case when workload variation is unbounded, it is costly to process a large job and let small jobs wait. In the next section we will introduce some job-priority based policies under the condition where workload variation is unbounded.

3 Polling System with Bounded Setup Times

In this section we mainly discuss the policies for the polling system with bounded setup times. Here we allow the workload variation to be unbounded. As shown in Theorem 6, policies in Π_r do not have constant competitive ratio if processing times are unbounded. It is because policies in Π_r are based on the cyclic routing discipline. When the setup times are bounded and workload variation is unbounded, an online policy may want to favor job priority instead of queue priority. For convenience of analysis, we assume switching time τ is bounded by a ratio of the minimal workload, that is $\tau \leq \theta p_{min}$. If $\tau = p_{min} = 0$, we let $\theta = 1$. Using the standard notation for scheduling problems from [18], we denote this polling problem as $1|r_i, \tau \leq \theta p_{min}| \sum C_i$. Notice when setup time is small, i.e., θ is small, switching may not be the major contributor to the completion time delay. We shall show how to define a 'small' θ in Section 4. This setting in the 3D printing example corresponds the scenario when jobs of a different color need to be printed, switch of inks is incurred and switching time is small compared to processing time. In this section, we mainly show several polices for one machine scheduling problem without switching time work well in the polling system when setup times are small, and we mainly discuss the policies that favor job priority more than queue priority by considering job processing times. Since job processing time is revealed upon arrival, we may want online policies to use job size information.

3.1 One Machine Policy and Gittins Index Policy for Polling system

In this subsection we mainly introduce two policies that favor jobs with short processing times. We first introduce a benchmark for deriving the competitive ratio for those policies. Usually the competitive ratio ρ is defined by $\sup_I \frac{C^\pi(I)}{C^*(I)} \leq \rho$ where $C^*(I)$ is the completion time for I in the offline optimal solution. However the offline problem is strongly NP-hard. To non-rigorously show the NP-hardness, we know if no preemption is allowed, even the easier problem $1|r_i| \sum C_i$ (without switching time) is strongly NP-hard [23, 19, 21]. Instead of using offline optimal solution to serve as the benchmark for online policies, in this section we mainly use a lower bound of the optimal solution as the benchmark. To get a lower bound for the optimal solution in the case with unbounded processing times and bounded setup times, we introduce the idea of setup time reduced instance. Suppose instance I is an arbitrary instance, the setup time reduced instance of I, say $\underset{\sim}{I}$, is an instance that has the same arrivals and workload as I, but with no setup time in $\underset{\sim}{I}$. The optimal scheduling policy for $\underset{\sim}{I}$ to minimize total completion times is to serve the job with *Shortest Remaining Processing Time* preemptively (SRPT) [37]. This schedule policy is also online, which is handy for online policies to emulate. The completion time of instance $\underset{\sim}{I}$ by SRPT is noted as $C^p(\underset{\sim}{I})$. Note that setup times are not counted in $\underset{\sim}{I}$, thus $C^p(\underset{\sim}{I}) \leq C^*(I)$. In our problem, we only consider the problem without

preemption, and we only use SRPT as the benchmark. When preemption is not allowed, *one machine (OM)* policy is proved to be the online scheduling policy with smallest competitive ratio for $\underset{\sim}{I}$ [32]. When setup time is small, we can adopt OM directly into I, regardless of the setup times. The description of OM is provided in Algorithm 4, and the competitive ratio of OM is provided in Theorem 11.

Algorithm 4 One Machine Scheduling (OM)

1. Simulate SRPT policy on the setup time reduced instance $\underset{\sim}{I}$.
2. Schedule the jobs non-preemptively in the order of completion time of jobs by SRPT on $\underset{\sim}{I}$.

Theorem 11. *OM is a* $(2+\theta)$*-competitive online algorithm for the polling system* $1|r_i, \tau \leq \theta p_{min}|\sum C_i$.

Proof. Let the completion time of the j^{th} job under OM scheduling be C_j^o, and the completion time of the j^{th} job completed under SRPT as C_j^p. Since job j is also the j^{th} job that completes service under SRPT, thus $\sum_{i=1}^{j} p_j \leq C_j^p$. Then we have $C_j^o \leq C_j^p + \sum_{i:C_i^p \leq C_j^p} p_i + j\tau \leq 2C_j^p + \sum_{i=1}^{j} p_i \leq (2+\theta)C_j^p$. Since $C^p(\underset{\sim}{I}) \leq C^*(I)$, we have $\sum C_i^o \leq (2+\theta)\sum C_i^p \leq (2+\theta)\sum C_i^*$. □

OM algorithm is intuitive, easy to apply and polynomial-time solvable. Despite its simplicity, we may find it inefficient since switching time is ignored. Although each of the unnecessary switch only brings a small amount of delay, we may still want to avoid switching too often. Thus we provide another policy which is based on Gittins Index. Gittins Index policy is a well-studied method for solving problems such as the multi-armed bandit problem [42, 16]. Gittins Index policy is also the optimal policy for the $M/G/1$ multi-class queue scheduling problem to minimize the mean average sojourn times [1]. Here we modify the Gittins Index policy and use it for the polling system with setup time by assigning indices to jobs and then choosing the best index. We call it the Gittins Index policy for polling system (GIPP), which is shown in Algorithm 5. We denote the completion time of job i in I by GIPP as C_i^g. The performance of GIPP is provided in Theorem 12.

Algorithm 5 Gittins Index Policy for Polling System (GIPP)

Require: Instance I
Denote the queue that the server is serving as $queue_{server}$
while I has not been fully processed **do**
    Simulate SRPT. Regard the departure time of the i^{th} job in SRPT as the i^{th} arrival time in GIPP. Denote the Gittins index of job i as $index_i$
    if i^{th} arrival is at $queue_{server}$ **then**
        $index_i = \frac{1}{p_i}$
    else
        $index_i = \frac{1}{p_i+\tau}$
    end if
    Schedule the available jobs by the largest index first
end while
return Total completion time $C^g(I)$

Theorem 12. *GIPP is a* $(2+\theta)$*-competitive online algorithm for* $1|r_i, \tau \leq \theta p_{min}|\sum C_i$.

Proof. Note both GIPP and OM simulate SRPT on $\underset{\sim}{I}$ and schedule job i only after job i has been processed in SRPT. So we can regard OM as the first come first service policy in a job instance whose arrival times are $\{C_i^p, i = 1, 2, ...\}$, while GIPP serves the job with the largest Gittins index first in this instance of arrival

times $\{C_i^p, i = 1, 2, ...\}$. We have $C_j^g \leq C_j^p + \sum_{i=1}^{j} \tilde{p}_i$, where $\tilde{p}_i$ is given by inverse of the Gittins index of job i. Since GIPP schedule the available jobs in the ascending order of $\tilde{p}_i$, we have $C_j^g \leq C_j^p + \sum_{i=1}^{j} \tilde{p}_i \leq C_j^p + \sum_{i:C_i^p \leq C_j^p} (p_i + \tau) \leq (2+\theta) C_j^p$. The competitive ratio is tight when there is only one job in instance I, available at time 0. Suppose this job has processing time 1. Then $C^p(I) = 1$, and $C^g(I) = 1 + (1+\theta) = 2+\theta$. Hence proved. □

Intuitively, GIPP might be better than OM since GIPP always schedules jobs of large indices and leaves jobs with small indices later. A job from a queue different from the server may have a small Gittins Index due to the setup time τ, thus GIPP may avoid switching frequently. Surprisingly however, it does not have a competitive ratio smaller than OM. Both OM and GIPP have the same competitive ratio (see Theorems 11 and 12), when using SRPT on reduced instance as the benchmark.

Next we show the lower bound competitive ratio of the problem $1|r_i, \tau = \theta p_{min}| \sum C_i$. Notice this is a special case for the problem $1|r_i, \tau \leq \theta p_{min}| \sum C_i$ if τ and p_{min} are both revealed deterministically. There is no online algorithm that has a competitive ratio smaller than the lower bound for $1|r_i, \tau = \theta p_{min}| \sum C_i$.

Theorem 13. *If $\tau = \theta p_{min}$ and $\theta \geq 0$, then there is no online algorithm whose competitive ratio is smaller than $\theta + 1$, using SRPT on the reduced instance as the benchmark.*

Proof. If there is one job of processing time p_{min} in the system we have $\frac{C^\pi(I)}{C^p(I)} \geq \frac{(1+\theta)p_{min}}{p_{min}} = 1 + \theta$. If $\tau = p_{min} = 0$, then the lower bounded ratio is $\theta + 1 = 2$ as provided in [20] since we assume $\theta = 1$ in this case. □

A natural question is whether this lower bound is the best lower bound that one can have. The answer remains open. There could be either online policy that performs exactly as the lower bound so that it is the best online policy, or a greater lower bound which is closer to the ratio $(2 + \theta)$.

3.2 Simulation-based Policies

In Section 2 we introduced the idea of simulation-based policies. A simulation-based policy usually simulates an online policy in parallel, and schedules jobs based on results of the simulated policy. In the case when setup times are bounded, we can also construct simulation-based policies, knowing that SRPT is the optimal policy for the problem without setup times. Given any instance I for the polling system, there is a reduced instance $\underset{\sim}{I}$ in which arrival times are the same as I but setup time is 0. There is also a setup time augmented instance $\tilde{I}$, where setup time is 0 but each workload is augmented with τ, i.e., $p_i \in I$ corresponds $p_i + \tau$ in $\tilde{I}$. Any online algorithm can simulate policies on $\tilde{I}$ and $\underset{\sim}{I}$ in parallel, and make decisions based on the simulation results. We have the following results via Corollaries 14 and 15.

Corollary 14. *By simulating any ρ-competitive online algorithm that we call σ on $\underset{\sim}{I}$ and scheduling jobs by the order of completion times under σ, we obtain an online algorithm on I that is $\rho(2 + \theta)$-competitive for $1|r_i, \tau \leq \theta p_{min}| \sum C_i$.*

Proof. Denote the new online algorithm on I as π. $C_j^\pi \leq C_j^\sigma + \sum_{i=1}^{j} p_i + j\tau \leq (2+\theta) C_j^\sigma$. Thus $\sum C_j^\pi \leq \sum (2+\theta) C_j^\sigma \leq \sum \rho(2+\theta) C_j^p$.

SRPT is the optimal policy on $\underset{\sim}{I}$ and it is online, thus it is $1-$competitive for $1 \mid r_i \mid \sum C_i$. By following the decision that SRPT makes we have a policy with competitive ratio $(2 + \theta)$, which is the same as the result

of Theorem 11. We can also simulate online policies on augmented instance $\tilde{I}$ and follow their decisions, by which we have the following corollary. □

Corollary 15. *By simulating a ρ-competitive online algorithm that we call σ on $\tilde{I}$ and scheduling jobs by the order of their completion times under σ, we obtain an online algorithm on I that is $2\rho(1+\theta)$-competitive for $1|r_i, \tau \leq \theta p_{min}|\sum C_i$.*

Proof. Denote the online algorithm on I as π. We first show that $C^p(\tilde{I}) \leq (1+\theta)C^p(\underset{\sim}{I})$. For an arbitrary job $\tilde{p_i} \in \tilde{I}$, it satisfies $\tilde{p_i} = p_i + \tau \leq (1+\theta)\underset{\sim}{p_i}$. Let $\tilde{\sigma}$ be a policy that works on $\tilde{I}$ but schedules jobs in the same order as SRPT on $\underset{\sim}{I}$, and serves each job the same portion as SRPT on $\underset{\sim}{I}$. Thus $C^p(\tilde{I}) \leq C^{\tilde{\sigma}}(\tilde{I}) \leq (1+\theta)C^p(\underset{\sim}{I})$. By $C_j^{\pi} \leq \tilde{C_j^{\sigma}} + \sum_{i=1}^{j} p_i + j\tau \leq 2\tilde{C_j^{\sigma}}$ we have

$$C^{\pi}(I) \leq 2C^{\sigma}(\tilde{I}) \leq 2\rho C^p(\tilde{I}) \leq 2\rho(1+\theta)C^p(\underset{\sim}{I}) \leq 2\rho(1+\theta)C^p(I).$$

Hence proved. □

Since the optimal policy for $\tilde{I}$ is SRPT, ρ in Corollary 14 is at least 1. By simulating SRPT on $\tilde{I}$ and following its decisions we have $C^{\pi}(I) \leq 2(1+\theta)C^p(I)$, which is greater than ratio $(2+\theta)$ that we get by simulating SRPT on $\underset{\sim}{I}$. Furthermore, if we simulate the same online policy based on either $\underset{\sim}{I}$ or $\tilde{I}$, we have $\rho(2+\theta) \leq 2\rho(1+\theta)$ for any $\rho \geq 1$, which implies that it is always better to simulate it on the reduced instance $\underset{\sim}{I}$.

3.3 Conditions for Constant Competitive Ratios

So far we have discussed bounded workload variation case (Section 2) and bounded setup times in this section. We then discuss the case when both setup time and workload variation are bounded. Obviously the OM and GIPP work under this scenario and have constant competitive ratios. The algorithms in Π_r which we provide in Section 2 also have constant competitive ratio in this case if number of queues is fixed. However, in this special case we have more policies with constant competitive ratios. In the following we show that all the work-conserving polices (means server never idles if the system is not empty) have a constant competitive ratio in this case.

Theorem 16. *Any non-preemptive work-conserving (WC) policy on the polling system with $p_{max} \leq \gamma p_{min}$ and $\tau \leq \theta p_{min}$ is at least $(\gamma+\theta)$-competitive with respect to the optimal solution to the offline problem.*

Proof. Let $\hat{I}$ be the workload and setup time augmented instance that all jobs are of workload $\hat{p} = p_{max} + \tau \leq (\gamma+\theta)p_{min}$, setup time is 0 in $\hat{I}$ and arrivals are the same as I. Note that any non-preemptive work-conserving policy on $\hat{I}$ is optimal since processing times for jobs in $\hat{I}$ are identical. Let σ be a non-preemptive work-conserving policy on I, and $\hat{\sigma}$ is a policy that works on $\hat{I}$ but makes the same decisions that σ makes. Then $\hat{\sigma}$ is work-conserving since σ never idles when there are unfinished jobs in system, and therefore $C^{\hat{\sigma}}(\hat{I}) = C^*(\hat{I})$. Now we show $C^*(\hat{I}) \leq (\gamma+\theta)C^*(I)$. Let $\tilde{S}_i^*$ be the starting time of job i in $\hat{I}$ under the optimal solution and S_i^* be the starting time of job i in I under the optimal solution. Let δ be a policy that works on $\hat{I}$ and finishes each job i at $(\gamma+\theta)C_i^*$. Notice that $(\gamma+\theta)C_i^* = (\gamma+\theta)(S_i^* + p_i) \geq (\gamma+\theta)S_i^* + \hat{p}$. Let W_i^* be the non-service times of the optimal policy on I from time 0 to S_i^*, i.e., sum of setup times and idling times till S_i. Then $(\gamma+\theta)S_i^* = (\gamma+\theta)(W_i + \sum_{j=1}^{i-1} p_j) \geq (\gamma+\theta)W_i + (i-1)\hat{p} \geq \hat{S}_i^*$. The last inequality holds

Assumption	p_{max}	τ	Competitive Ratio
Bounded Workload and Setup Time	$p_{max} \leq \gamma p_{min}$	$\tau \leq \theta p_{min}$	$\min\{2+\theta, \gamma+\theta, \max\{\frac{3}{2}\gamma, k+1)\}\}$
Bounded Setup Time	N/A	$\tau \leq \theta p_{min}$	$2+\theta$
Bounded Workload	$p_{max} \leq \gamma p_{min}$	N/A	$\max\{\frac{3}{2}\gamma, k+1\}$
Unbounded Workload and Setup Time	N/A	N/A	≥ 2

Table 3: Competitive Ratios for Different Cases

because the optimal policy on $\hat{I}$ is work-conserving. Thus we have shown that δ is a feasible schedule on $\hat{I}$. In summary, we have

$$C^{\sigma}(I) \leq C^{\hat{\sigma}}(\hat{I}) = C^{*}(\hat{I}) \leq C^{\delta}(\hat{I}) = (\gamma+\theta)C^{*}(I).$$

Hence proved. □

So far we have shown the existence of constant competitive ratios under different assumptions for polling systems. Table 3 summarizes the competitive ratios that we prove in this paper. We also provide the following theorem to summarize the necessary and sufficient conditions to result in constant competitive ratio for the general polling system.

Theorem 17. *The sufficient conditions for existence of constant competitive ratio is when either of the following condition holds,*

(1) $\tau \leq \theta p_{min}$ for some constant θ;

(2) $p_{max} \leq \gamma p_{min}$ for some constant γ.

One of the necessary conditions is given by: There is no online policy with competitive ratio smaller than k for $1 \mid r_i, \tau \mid \sum C_i$ if its routing discipline is static or random, or is purely queue-length based, or purely job-priority based.

Proof. Proof for sufficient conditions directly follows from Theorems 1 and 11. Proof for the necessary condition follows from Theorem 5, which shows no policy with static or random routing discipline has competitive ratio smaller than k. To show it holds for and queue-length based routing policy π_1, we give an instance I where there is one job at each queue at time 0, with the job in queue 1 having workload p and jobs in other queues having workload 0. Since π_1 is purely queue-length based, it treats all queues equally since the queue lengths are equal, so we suppose the server serves from queue 1 to queue k, Thus

$$C^{\pi_1}(I) \quad = \quad p + (k-1)p + \tau\frac{k(k+1)}{2},$$

and

$$C^{*}(I) \quad = \quad p + \tau\frac{k(k+1)}{2}.$$

Letting $p \to \infty$, we have $\frac{C^{\pi_1}(I)}{C^{*}(I)} = k$.

To show the result holds for any purely job-priority based policy π_2, we give an instance $I^{'}$ where there are n jobs at queue 1 and one job at queue $i = 2, ..., k$ at time 0, with all jobs having workload 0. Since π_2 is purely job priority based, it does not consider any queue information. Notice both OM and GIPP are purely job priority based. Suppose π_2 serves $I^{'}$ from queue k to queue 1, thus

$$C^{\pi_2}(I^{'}) \quad = \quad \tau\left((n+k-1)+(n+k-2)+...+n\right),$$

and

$$C^{*}(I^{'}) \quad = \quad \tau\left((n+k-1)+(k-1)+...+1\right).$$

Letting $n \to \infty$, we have $\frac{C^{\pi_2}(I^{'})}{C^{*}(I^{'})} = k$. Hence proved. □

We should note that those two sufficient conditions are not trivial at all. In fact in many instances either the workload (processing time) is bounded or the setup time is bounded or both, where constant competitive ratio algorithms exist. Theorem 5 says the lower bound competitive ratio is given by 2, however Theorem 17 says that if an online policy has purely static or random routing policy, or purely queue-length based routing policy or purely job-priority based routing policy, the best achievable competitive ratio is $k \geq 2$. This implies if one online policy wants to achieve a competitive ratio smaller than k, a novel routing discipline which combines queue priority and job priority is needed. We shall introduce a mixed strategy which can practically reduce the competitive ratio in Section 4.

3.4 Clearing Problem

In Section 3.1 we show both OM and GIPP are $(2+\theta)$ competitive, however the lower bound competitive ratio is $(1+\theta)$ as shown in Theorem 13. This means either there exists a better algorithm with competitive ratio smaller than $(2+\theta)$ or the actual lower bound is greater than $(1+\theta)$. However, in this subsection we show the lower bound $(1+\theta)$ is not trivial for the clearing problem. Note that the clearing problem is a special case of the online problem, with all arrivals happen at time 0. We show in this subsection that for the clearing problem, both of OM and GIPP have a tight competitive ratio $(1+\theta)$. We denote the clearing problem as $1|r_i = 0, \tau \leq \theta p_{min}|\sum C_i$. Moreover, since there is no arrival in the future, both OM and GIPP do not need to simulate SRPT in parallel. We thus modify OM and GIPP, by truncating the simulation part, then OM schedules jobs by smallest workload first and GIPP schedules jobs by the largest Gittins index first.

Theorem 18. *OM and GIPP both have competitive ratio $\theta + 1$ for $1|r_i = 0, \tau \leq \theta p_{min}|\sum C_i$.*

Proof. There is no idling time in schedule C^o. Suppose jobs are indexed in ascending workload order so that $p_1^p \leq p_2^p \leq ... \leq p_n^p$, then $C_j^o \leq \sum_{i=1}^{j} p_i^p + \tau j$. Since $C_j^p = \sum_{i=1}^{j} p_i^p$ and $\tau \leq \theta p_{min}$, we have $C_j^o \leq (1+\theta)C_j^p$, thus $C^o(I) \leq (1+\theta)C^p(I)$.

We now follow jobs scheduled by GIPP. Let p_i^g be the inverse of Gittins index for i^{th} job scheduled by GIPP. If p_i^p is available for GIPP to schedule, then we have $p_i^g \leq p_i^p + \tau$. Now we consider the case in which p_i^p is not available to GIPP as GIPP serves p_i^p before p_i^g. Let $\underset{\sim}{S}(i)$ be the set of jobs served by SRPT but not by GIPP after the i^{th} schedule. At time C_{i-1}^g, if $\underset{\sim}{S}(i-1) = \emptyset$, then p_i^p is available for GIPP to schedule. Thus

we assume $S(i-1)$ is non-empty. Choose an arbitrary job p^* from $S(i-1)$ we have $p_i^g \leq p^* + \tau \leq p_i^p + \tau$, so that $p_i^g \leq p_i^p + \tau$. Therefore $C_j^g = \sum_{i=1}^{j} p_i^g \leq \sum_{i=1}^{j} p_i^p + j\tau$ and $C^g(I) \leq (1+\theta)C^p(I)$. □

We know the clearing problem with fixed number of queues is polynomial time solvable [29, 15]. The problem with dynamic number of queues is solvable using a 2-approximating algorithm provided in [9]. However here we say that if $p_{min} > 0$ and τ is bounded, simple algorithms exist for the clearing problem and their competitive ratios are proved to be tight and optimal. The results look worse than competitive ratio 2, however here we use SRPT as the benchmark, which means the actual competitive ratio could be smaller than $\theta + 1$. The work [9] does not show the tightness of algorithms. Before presents a mixed strategy in Section 4, we next present algorithms for the offline algorithm.

3.5 On Approximating Algorithms for the Generalized Offline Problem

In this subsection we discuss the case in which job information is available to the server at time 0 but arrival times in the future are known, unlike the clearing problem where all jobs are available at time 0. Because the focus of this paper is mainly on solving the online scheduling problem, we do not wish to discuss the offline problem in detail. The main purpose of this subsection is to model the offline problem and provide an approximation method as well. To make the formulation more comprehensive and general, we add another constraint known as *precedence*, through the use of priorities. If job j has higher priority than job i, we note $j \succ i$ and job i cannot be served before j. We suppose each job i has a weight w_i, and we want to minimize the total weighted completion times. This problem is NP-hard [19]. For the convenience of notation, we denote this problem as $1|r_i, \tau, prec|\sum w_i C_i$ and formulate it in the following manner, where ξ_i is the queue where job i arrives:

$$
\begin{aligned}
\min \quad & \sum_{i=1}^{n} w_i C_i & & \\
s.t. \quad & C_i \geq r_i + p_i & \text{for } i = 1, ..., n, & (3.1)\\
& C_j \geq C_i + p_i & \text{for all } j \succ i \text{ and } i = 1, ..., n & (3.2)\\
& C_i \geq C_j + p_j + \tau 1_{\xi_i \neq \xi_j} \text{ or } C_j \geq C_i + p_i + \tau 1_{\xi_i \neq \xi_j} & \text{for } \textit{each pair } (i,j). & (3.3)
\end{aligned}
$$

Notice the constraint (3.3) is non-linear, we relax it by using the method provided in [19]. We thus have

$$\sum_{i=1}^{n} p_i C_i \geq \sum_{j=1}^{n} p_j (\sum_{i=1}^{j} p_i) = \frac{1}{2}\left(p(I)^2 + p^2(I)\right), \tag{3.4}$$

where $p(I) = \sum_{i=1}^{n} p_i$ and $p^2(I) = \sum_{i=1}^{n} p_i^2$.
We extend Inequality (3.4) to every subset of instance I, so that for arbitrary $S \subseteq I$ we have

$$\sum_{i \in S} p_i C_i \quad \geq \quad \frac{1}{2}\left(p(S)^2 + p^2(S)\right). \tag{3.5}$$

Notice the linear problem which minimizes $\sum w_i C_i$ with only constraints (3.1), (3.2) and (3.5) is solvable in

polynomial time via the ellipsoid algorithm [33, 19], we obtain the optimal solution to this linear problem as $\bar{C_1}, ..., \bar{C_n}$, and then schedule the jobs in order of non-decreasing $\bar{C_i}$. We note this schedule as *Schedule by* $\bar{C_i}$.

Theorem 19. *The Schedule by* $\bar{C_i}$ *is a* $(3+2\theta)-$*competitive algorithm for* $1|r_i, \tau \leq \theta p_{min}, prec| \sum w_i C_i$.

Proof. We let $\tilde{C_i}$ be the actual completion time for job i by following *schedule by* $\bar{C_i}$. We assume $\bar{C_1} \leq ... \leq \bar{C_n}$ are completion times on solving $\min \sum w_i C_i$ under constraints (3.1), (3.2) and (3.5), then for any $S = \{1, 2, ..., j\}$ we have

$$\bar{C_j} \sum_{i=1}^{j} p_i \geq \sum_{i=1}^{j} \bar{C_i} p_i \geq \frac{1}{2} \left(p(S)^2 + p^2(S) \right).$$

Since $\tilde{C_j} \leq \max_{i=1}^{j} r_i + \sum_{i=1}^{j} p_i + j\tau$ and $\max_{i=1}^{j} r_i \leq \bar{C_j}$, therefore $\tilde{C_j} \leq (3+2\theta)\bar{C_j}$. Suppose C_i^* is the optimal completion time for job i in the original problem with constraints (3.1) and (3.3), because constraint (3.5) is a relaxation of (3.3), then $\sum w_i \bar{C_i} \leq \sum w_i C_i^*$. Thus $\sum w_i \tilde{C_i} \leq (3+2\theta) \sum w_i C_i^*$. □

This competitive ratio looks worse than the ratio $2+\theta$ achieved by online policies, however we should notice in this offline problem we have precedence constraints. This also points to the problem with precedence is harder than the one without. The discussion for the offline precedence and release time constrained scheduling problem can be found in [11, 32, 19], but none of them consider a setup time constraint.

4 A Mixed Strategy

In Section 2 we provided a policy set Π_r. Policies in Π_r are based on cyclic routing discipline, and they have competitive ratio κ when workload variation is bounded. When workload variation is unbounded, we may want to use GIPP, with competitive ratio $\theta + 2$. In reality, it is common to see cases where workload variation is unbounded, and setup times are large but bounded. For example in reconfigurable manufacturing, robust components are used to process customized jobs and also designed to reduce setup times to make the manufacturing system more efficient. It is very rare to see unbounded setup times [28]. In 3D printing example, jobs are usually highly customized and heterogeneous, thus may have a large workload variation. Besides, when jobs from a very different prototype is received, many setup steps need to be performed and setting up the 3D printer would take a large amount of time, but the setup time is usually bounded. Thus we are interested in the problem where setup time is large but bounded, and workload variation is unbounded. From Section 3 we know the online problem can be solved by OM or GIPP, however, if a large workload is rare, we may want to adopt a policy from Π_r most of time if it gives a competitive ratio smaller than $\theta + 2$. This motivates us to construct a mixed strategy such that if no workload is greater than a threshold, say η, then a policy from Π_r is applied, resulting in a competitive ratio $\max\{\frac{3}{2}\eta, k+1\}$; if there is a new arrival with workload greater than η, then GIPP is applied so competitive ratio is $\theta+2$. If $\kappa(\eta) = \max\{\frac{3}{2}\eta, k+1\} \leq \theta+2$, this mixed strategy results in a smaller competitive ratio than simply using GIPP. We select the exhaustive policy π^e which does not wait once the queue is exhausted and skips empty queues from Π_r. Within a queue we let π^e adopt *Shortest Processing Time First* (SPT). It helps reduce the queue length within each queue, though this does not reflect on the competitive ratio. A formal statement of this mixed strategy π^m is provided in Algorithm 6.

Algorithm 6 Deterministic Mixed Strategy π^m

while The system is not empty **do**
  Use π^e: When a service is done, serve the next job with shortest processing time first; switch to the next non-empty queue when the current queue has been exhausted
  if There is an arrival whose workload is greater than η **then**
    if The server is serving **then**
      Finish the current job
    else if The server is setting up **then**
      Halt setting up and the server stays in the current queue
    end if
    Use GIPP
  end if
end while
return Total completion time $C^{\pi_m}(I)$

Theorem 20. *Suppose $p_{min} > 0$, $\eta \leq \theta$ and workload is drawn from a distribution, then*

$$\sup \frac{C^{\pi^m}(I)}{C^*(I)} \quad \leq \quad \nu(\eta),$$

where $\nu(\eta) = \kappa(\eta)\mu(\eta)^{n(I)} + (\theta+2)(1-\mu(\eta)^{n(I)})$ and $\mu(\eta) = \mathbf{P}(p_i \leq \eta)$, prior to revealing.

Proof. Let $p_{min} = 1$. If all the jobs are of workload smaller than η, then throughout the busy period, $\pi^m = \pi^e$ and $\frac{C^{\pi^m}(I)}{C^*(I)} \leq \kappa(\eta)$. Now we show that if there is a workload in the busy period with workload greater than η, $\frac{C^{\pi^m}(I)}{C^*(I)} \leq \theta + 2$. Suppose $I = I_1 \cup I_2$, where I_1 is the job instance that is served by π^e in I, and I_2 is the rest of I which is served by GIPP. Let S_2 be the time when I_2 starts service. Then $C^{\pi^m}(I) = C^{\pi^e}(I_1) + S_2 n(I_2) + C^g(I_2)$ where $C^g(I_2)$ is the completion time for GIPP that starts at time S_2. For the optimal solution, we have $C^*(I) = C^*(I_1 \cup I_2) \geq C^*(I_1) + R_2 n(I_2) + C^p(I_2)$, where R_2 is the time when the job with workload greater than η arrives and $C^p(I_2)$ is the completion time for I_2 under SRPT. Since $R_2 \leq S_2 \leq R_2 + \max\{\theta, \eta\} = R_2 + \theta$ by $\eta \leq \theta$, we have,

$$\frac{C^{\pi^m}(I)}{C^*(I)} \leq \frac{C^{\pi^e}(I_1) + S_2 n(I_2) + C^g(I_2)}{C^*(I_1) + R_2 n(I_2) + C^p(I_2)} \leq \frac{C^{\pi^e}(I_1) + (R_2+\theta) n(I_2) + C^g(I_2)}{C^*(I_1) + R_2 n(I_2) + C^p(I_2)}.$$

If $R_2 \geq \theta$, then we have $\frac{C^{\pi^m}(I)}{C^*(I)} \leq \max\{\kappa, 2, \theta+2\} = \theta + 2$. If $R_2 < \theta$, then $S_2 = \theta$ and

$$\frac{C^{\pi^m}(I)}{C^*(I)} \leq \frac{C^g(I_2)}{C^p(I_2)} \leq \theta + 2.$$

Hence proved. □

Thus the Deterministic Mixed Strategy has a competitive ratio smaller than purely using policies from Π_r or GIPP since $\nu(\eta)$ is a convex combination of $\kappa(\eta)$ and $(\theta+2)$, given $\kappa(\eta) \leq (\theta+2)$. Note that $\nu(\eta) = (\theta+2) + \mu(\eta)^{n(I)}(\kappa(\eta) - (\theta+2))$. If $\frac{3}{2}\eta \leq k+1$, then $\kappa(\eta) = k+1$. The optimal η in this case is given by $\frac{2}{3}(k+1)$ given $\mu(\eta)$ is an increasing function of η. If $\frac{3}{2}\eta \geq k+1$, then $\nu(\eta) = (\theta+2) + \mu(\eta)^{n(I)}(\frac{3}{2}\eta - (\theta+2))$, and the optimal value η^* can be obtained by solving

$$\begin{aligned} \min \quad & (\theta+2)+\mu(\eta)^{n(I)}(\tfrac{3}{2}\eta-(\theta+2)) \\ s.t. \quad & \tfrac{2}{3}(k+1)\le\eta\le\tfrac{2}{3}(\theta+2). \end{aligned} \tag{4.1}$$

Notice $\frac{2}{3}(k+1)$ is a feasible solution to System (4.1). Thus by solving System (4.1) we can obtain the optimal solution to $\nu(\eta)$, which is η^*. If we randomize the Deterministic Mixed Strategy by letting η be a random variable with mean η^*, we get the *Randomized Mixed Strategy.* We will see the Randomized Mixed Strategy gets a smaller competitive ratio than η^*.

Corollary 21. *If $\mu(\eta)^{n(I)}(\theta+2-\frac{3}{2}\eta)$ is convex then we can get a Randomized Mixed Strategy with competitive ratio smaller than the Deterministic Mixed Strategy in Algorithm 6.*

Proof. The competitive ratio for the Deterministic Mixed Strategy is given by $\nu(\eta)=(\theta+2)-\mu(\eta^*)^{n(I)}(\theta+2-\frac{3}{2}\eta^*)$ where η^* is the optimal solution to System (4.1). We let η be a random variable with mean η^*. The competitive ratio for this randomized policy is given by $\sup\frac{\mathbf{E}[C^{\pi^m}(I)]}{C^*(I)}\le\mathbf{E}[\nu(\eta)]=\mathbf{E}[(\theta+2)-\mu(\eta)^{n(I)}(\theta+2-\frac{3}{2}\eta)]$ where $\theta+2\ge\frac{3}{2}\eta$. Since $\mu(\eta)^{n(I)}(\theta+2-\frac{3}{2}\eta)$ is convex, we have $\mathbf{E}[(\theta+2)-\mu(\eta)^{n(I)}(\theta+2-\frac{3}{2}\eta)]\le\theta+2-\mu(\eta^*)^{n(I)}(\theta+2-\frac{3}{2}\eta^*)=\nu(\eta^*)$. □

Both Randomized Mix Strategy and Deterministic Mix Strategy provide competitive ratios smaller than $\max\{\kappa,\theta+2\}$, however to get the optimal threshold η, the number of jobs $n(I)$ in a busy period is needed. In the online problem where the information of future jobs is unknown to the server, the server cannot actually optimize the System (4.1). A practical way is letting $\eta=\frac{2}{3}(k+1)$ in the Deterministic Mixed Strategy, which eventually results in a competitive ratio smaller than $\theta+2$. However, both of these strategies give insights of revealing information, which could be our future work. If one can reveal or estimate the number of jobs in a busy period, then a System (4.1) is solvable thus Randomized Mixed Strategy is applicable and a smaller competitive ratio can be obtained.

5 Concluding Remarks and Future Works

In this paper we firstly consider polling systems by worst case analysis. Our analysis provides a novel way to classify scheduling policies for polling systems by considering their worst case performance without any stochastic assumptions. It allows to describe the performance of some policies even their average performance is intractable. Necessary and sufficient conditions for the existence of constant competitive ratio are provided and the worst case performance for several well-studied polling system scheduling policies are discussed. We show that both cyclic exhaustive policy and cyclic gated policy have constant competitive ratio κ for problem $1\mid r_i,\tau,p_{max}\le\gamma p_{min}\mid\sum C_i$, but they do not have constant competitive ratio for a general problem $1\mid r_i,\tau\mid\sum C_i$. Interestingly, we find cyclic is the optimal static routing discipline, and when cyclic routing discipline is adopted, revealing the processing times for jobs is not helpful in reducing the competitive ratios if $\frac{3}{2}\gamma\le k+1$. We also find some polices with provable average performance do not have constant competitive ratios for $1\mid r_i,\tau,p_{max}\le\gamma p_{min}\mid\sum C_i$ such as l-limit policy and SLQ policy. We also provide online policies that balance well the future uncertainty and current information availability, such as GIPP. Besides, we show that if the routing discipline for an online policy relies only on static or random routing table, or queue-length or job processing times, then the competitive ratio of this policy is at least k. We thus

provide a mixed strategy which can reduce the competitive ratio from using static routing discipline or GIPP only. Our analysis suggests a policy with competitive ratio smaller than k may need to incorporate more information than only considering job processing times or number of jobs in queue. However, the question that whether there exists an online policy with constant competitive ratio for the problem $1 \mid r_i, \tau \mid \sum C_i$ without any bound conditions for workload and setup times remains open. Also, it is unclear if there exists a better lower bound competitive ratio for all the online policies. A future problem to consider includes searching for online policies with smaller competitive ratios and deriving a better competitive ratio lower bound for all the online policies.

References

[1] Samuli Aalto, Urtzi Ayesta, and Rhonda Righter. On the gittins index in the M/G/1 queue. *Queueing Systems*, 63(1-4):437, 2009.

[2] Ali Allahverdi, CT Ng, TC Edwin Cheng, and Mikhail Y Kovalyov. A survey of scheduling problems with setup times or costs. *European journal of operational research*, 187(3):985–1032, 2008.

[3] Eitan Altman, Asad Khamisy, and Uri Yechiali. On elevator polling with globally gated regime. *Queueing Systems*, 11(1):85–90, 1992.

[4] Edward J Anderson and Chris N Potts. Online scheduling of a single machine to minimize total weighted completion time. *Mathematics of Operations Research*, 29(3):686–697, 2004.

[5] Josephe Baker and Izhak Rubin. Polling with a general-service order table. *IEEE Transactions on Communications*, 35(3):283–288, 1987.

[6] Nikhil Bansal, Bart Kamphorst, and Bert Zwart. Achievable performance of blind policies in heavy traffic. *Mathematics of Operations Research*, 2018.

[7] Marko AA Boon, RD Van der Mei, and Erik MM Winands. Applications of polling systems. *Surveys in Operations Research and Management Science*, 16(2):67–82, 2011.

[8] Chandra Chekuri, Rajeev Motwani, Balas Natarajan, and Clifford Stein. Approximation techniques for average completion time scheduling. *SIAM Journal on Computing*, 31(1):146–166, 2001.

[9] Srikrishnan Divakaran and Michael Saks. Approximation algorithms for problems in scheduling with set-ups. *Discrete Applied Mathematics*, 156(5):719–729, 2008.

[10] Srikrishnan Divakaran and Michael Saks. An online algorithm for a problem in scheduling with set-ups and release times. *Algorithmica*, 60(2):301–315, 2011.

[11] Jianzhong Du, Joseph Y-T Leung, and Gilbert H Young. Minimizing mean flow time with release time constraint. *Theoretical Computer Science*, 75(3):347–355, 1990.

[12] Leah Epstein and Rob van Stee. Lower bounds for on-line single-machine scheduling. *Theoretical Computer Science*, 299(1):439–450, 2003.

[13] M Ferguson and Y Aminetzah. Exact results for nonsymmetric token ring systems. *IEEE Transactions on Communications*, 33(3):223–231, 1985.

[14] Natarajan Gautam. *Analysis of queues: methods and applications*. CRC Press, 2012.

[15] Jay B Ghosh. Batch scheduling to minimize total completion time. *Operations Research Letters*, 16(5):271–275, 1994.

[16] John Gittins, Kevin Glazebrook, and Richard Weber. *Multi-armed bandit allocation indices*. John Wiley & Sons, 2011.

[17] Michel X Goemans, Maurice Queyranne, Andreas S Schulz, Martin Skutella, and Yaoguang Wang. Single machine scheduling with release dates. *SIAM Journal on Discrete Mathematics*, 15(2):165–192, 2002.

[18] Ronald L Graham, Eugene L Lawler, Jan Karel Lenstra, and AHG Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of discrete mathematics*, 5:287–326, 1979.

[19] Leslie A Hall, Andreas S Schulz, David B Shmoys, and Joel Wein. Scheduling to minimize average completion time: Off-line and on-line approximation algorithms. *Mathematics of operations research*, 22(3):513–544, 1997.

[20] Johannes Adzer Hoogeveen and Arjen PA Vestjens. Optimal on-line algorithms for single-machine scheduling. In *International Conference on Integer Programming and Combinatorial Optimization*, pages 404–414. Springer, 1996.

[21] AHG Rinnooy Kan. *Machine scheduling problems: classification, complexity and computations*. Springer Science & Business Media, 2012.

[22] Leonard Kleinrock and Hanoch Levy. The analysis of random polling systems. *Operations Research*, 36(5):716–732, 1988.

[23] Eugene L Lawler, Jan Karel Lenstra, Alexander HG Rinnooy Kan, and David B Shmoys. Sequencing and scheduling: Algorithms and complexity. *Handbooks in operations research and management science*, 4:445–522, 1993.

[24] Jan Karel Lenstra, David B Shmoys, and Eva Tardos. Approximation algorithms for scheduling unrelated parallel machines. *Mathematical programming*, 46(1-3):259–271, 1990.

[25] Zhen Liu, Philippe Nain, and Don Towsley. On optimal polling policies. *Queueing Systems*, 11(1-2):59–83, 1992.

[26] Xiwen Lu, RA Sitters, and Leen Stougie. A class of on-line scheduling algorithms to minimize total completion time. *Operations Research Letters*, 31(3):232–236, 2003.

[27] Elisabeth Lübbecke, Olaf Maurer, Nicole Megow, and Andreas Wiese. A new approach to online scheduling: Approximating the optimal competitive ratio. *ACM Transactions on Algorithms (TALG)*, 13(1):15, 2016.

[28] Mostafa G Mehrabi, A Galip Ulsoy, and Yoram Koren. Reconfigurable manufacturing systems: Key to future manufacturing. *Journal of Intelligent manufacturing*, 11(4):403–419, 2000.

[29] Clyde L Monma and Chris N Potts. On the complexity of scheduling with batch setup times. *Operations research*, 37(5):798–804, 1989.

[30] Gur Mosheiov, Daniel Oron, and Yaacov Ritov. Minimizing flow-time on a single machine with integer batch sizes. *Operations Research Letters*, 33(5):497–501, 2005.

[31] CT Ng, TC Edwin Cheng, JJ Yuan, and ZH Liu. On the single machine serial batching scheduling problem to minimize total completion time with precedence constraints, release dates and identical processing times. *Operations Research Letters*, 31(4):323–326, 2003.

[32] Cynthia Phillips, Clifford Stein, and Joel Wein. Minimizing average completion time in the presence of release dates. *Mathematical Programming*, 82(1-2):199–223, 1998.

[33] Maurice Queyranne. Structure of a simple scheduling polyhedron. *Mathematical Programming*, 58(1):263–285, 1993.

[34] Debashish Sarkar and WI Zangwill. Expected waiting time for nonsymmetric cyclic queueing systems exact results and applications. *Management Science*, 35(12):1463–1474, 1989.

[35] Andreas S Schulz and Martin Skutella. The power of α-points in preemptive single machine scheduling. *Journal of Scheduling*, 5(2):121–133, 2002.

[36] René Sitters. Competitive analysis of preemptive single-machine scheduling. *Operations Research Letters*, 38(6):585–588, 2010.

[37] Donald R Smith. A new proof of the optimality of the shortest remaining processing time discipline. *Operations Research*, 26(1):197–199, 1978.

[38] Leen Stougie and Arjen PA Vestjens. Randomized algorithms for on-line scheduling problems: how low can't you go? *Operations Research Letters*, 30(2):89–96, 2002.

[39] Hideaki Takagi. Queuing analysis of polling models. *ACM Computing Surveys (CSUR)*, 20(1):5–28, 1988.

[40] Jiping Tao, Zhijun Chao, and Yugeng Xi. A semi-online algorithm and its competitive analysis for a single machine scheduling problem with bounded processing times. *Journal of Industrial and Management Optimization*, 6(2):269–282, 2010.

[41] Jiping Tao, Zhijun Chao, Yugeng Xi, and Ye Tao. An optimal semi-online algorithm for a single machine scheduling problem with bounded processing time. *Information Processing Letters*, 110(8-9):325–330, 2010.

[42] John N Tsitsiklis. A short proof of the Gittins index theorem. *The Annals of Applied Probability*, 4(1):194–199, 1994.

[43] VM Vishnevskii and OV Semenova. Mathematical methods to study the polling systems. *Automation and Remote Control*, 67(2):173–220, 2006.

[44] Adam Wierman, Erik MM Winands, and Onno J Boxma. Scheduling in polling systems. *Performance Evaluation*, 64(9):1009–1028, 2007.

[45] Adam Wierman and Bert Zwart. Is tail-optimal scheduling possible? *Operations research*, 60(5):1249–1257, 2012.

[46] Erik MM Winands, Ivo Jean-Baptiste François Adan, and Geert-Jan van Houtum. Mean value analysis for polling systems. *Queueing Systems*, 54(1):35–44, 2006.

[47] Lele Zhang and Andrew Wirth. Online machine scheduling with family setups. *Asia-Pacific Journal of Operational Research*, 33(04):1650027, 2016.

A Proof for Theorem 1 in the Main Paper

In this section we mainly provide the proof for Theorem 1 of our original paper. Since the server in the online policy serves queues in a cyclic way, without loss of generality, we assume the server starts serving from queue 1. A cycle (round) starts when the server visits queue 1 and ends when it visits queue 1 the next time. If at some point a queue, say queue i, is empty and skipped by the server, we still regard queue i to be visited in the cycle, with setup time 0. We define the job instance served by the server in w^{th} visit to queue as b_i^w (for $i = 1, ..., k$), and we call it a *batch*. It is a subset of job instance I. In each cycle, there are k batches served by the online policy, some of them may be empty but not all of them (if all of them are empty then the system is empty and the server would idle at queue 1). We let $I^w = \cup_{i=1}^{k} b_i^w$. For each batch b_i^w with number of jobs $n(b_i^w) = n_i^w$, S_i^w is the earliest time when a job in b_i^w starts being processed under the online policy, and R_i^w is the earliest release date (arrival time) over all jobs in batch b_i^w. Notice that $R_i^w \leq S_i^w$. Each batch b_i^w may be processed by the optimal offline policy in a different way than the online policy. Suppose E_i^w is the earliest time when a job in batch b_i^w starts being served by the optimal offline policy. Note E_i^w may differ from S_i^w. Before time S_i^w, we know all batches b_l^h with $h < w$ for all $l = 1, ..., k$ and $h = w$ for $l = 1, ..., i-1$ (i.e., the set of instances $(\cup_{j=1}^{w-1} I^j) \cup (\cup_{l=1}^{i-1} b_l^w)$) have been served by the online policy. However in the optimal offline policy, only some jobs in these batches may have been served. We suppose q_i^w number of jobs in $(\cup_{j=1}^{w-1} I^j) \cup (\cup_{l=1}^{i-1} b_l^w)$ have been served by the optimal offline policy before time E_i^w. Note that q_i^w only denotes a number instead of a job instance. We let $C^\pi(I)$ be the cumulative completion times of all jobs in job instance I under online policy π, and $C^*(I)$ be the cumulative completion times of all jobs in I under the offline optimal policy. In Theorem 1 we want to show $\sup_I \frac{C^\pi(I)}{C^*(I)} \leq \rho$ for some constant ρ. If there is at least one job in the system, we say the system is busy, otherwise it is empty. When the system is empty, the server is not serving under the online policy. The status of the system under the online policy can be described as a busy period following an empty period, and then following by a busy period and so on. Since no jobs are served during empty periods, if we show in every busy period, the instance I that is served by the online policy satisfies $\frac{C^\pi(I)}{C^*(I)} \leq \rho$, then $\sup_I \frac{C^\pi(I)}{C^*(I)} \leq \rho$ holds for all I. There two types of busy periods that we are interested in. Type I busy period (denoted as I-B) is the busy period in which the server under online policy resumes work without setting up, which is because the server was idling at the last queue it served in the previous busy period, say queue i, and the first arrival in the new busy period also occurs at queue i. Type II busy period (denote as II-B) is the busy period in which the server resumes work with a setting up, which is because the new arrival occurs at a queue different from the queue that the server was idling at. We will consider these two types of busy period separately in the proof. For convenience, we let $g(I) = \frac{n(I)(n(I)+1)}{2}$ for job instance I, which is the sum of arithmetic sequence from 1 to $n(I)$. Also, $g(I)$ can be regarded as the completion time of I when all the jobs are available at time 0, each of them are of processing time 1, and no setting up time is considered. All the notations are summarized in

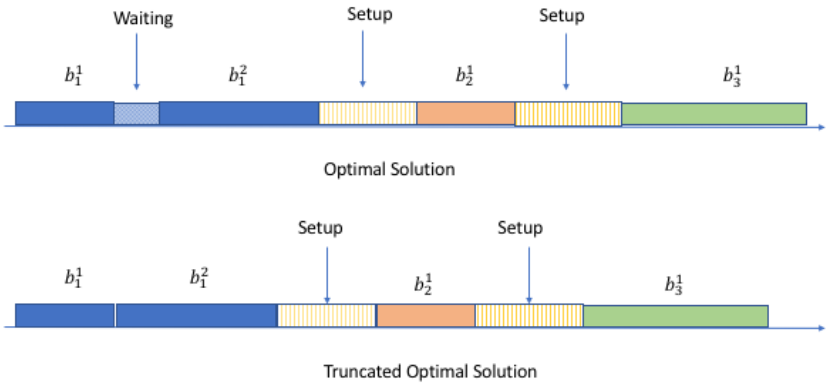

Figure A.1: Truncated Optimal Solution

Table 4 of this document. Before going to the proof of Theorem 1, we first provide an example to show how the total completion time is characterized.

Example 22. Suppose there is a job instance I of $n(I) = n_1 + n_2$ jobs which arrives at time R. Each of the job has processing time 1. The server starts to process the first n_1 jobs, and idles for time W, and then processes the rest of n_2 jobs without idling. The total completion time for I is given by

$$\begin{aligned} C^{\pi}(I) &= (R+1)+(R+2)+...+(R+n_1)+(R+n_1+W+1)+(R+n_1+W+2)+...+(R+n_1+W+n_2) \\ &= (n_1+n_2)R+n_2W+g(I). \end{aligned}$$

Notice the completion time of I is made up of three components. The first time $(n_1 + n_2)R$ is because all the jobs in I wait for time R before getting served. The second term n_2W is because the rest n_2 jobs wait for another W time. The third term $g(I)$ is the pure completion time if we process jobs one by one without idling.

Having showed the idea of calculating total completion times in Example 22, now we move on to show the proof of Theorem 1. We first introduce the idea of *truncated optimal schedule*. For a busy period in online scheduling, the corresponding optimal policy may not be work-conserving (i.e., never idles when there are jobs in the system). The optimal policy may wait at some point in order to receive more jobs. The truncated optimal solution is defined by the optimal completion time for the offline problem with subtracting the completion time caused by waiting, which is shown in Figure A.1. There is a waiting period W between b_1^1 and b_1^2 in Figure A.1. The truncated optimal solution is given by $C^*(b_1^1 \cup b_1^2 \cup b_2^1 \cup b_3^1) - W(n_1^1 + n_1^2 + n_2^1 + b_3^1)$. For simplicity, in the rest of this document when we say optimal solution to the offline problem, we refer to the truncated optimal solution. We also use $C^*(I)$ to denote the truncated optimal solution to instance I. Note that the truncated optimal solution is always a lower bound for the real optimal solution.

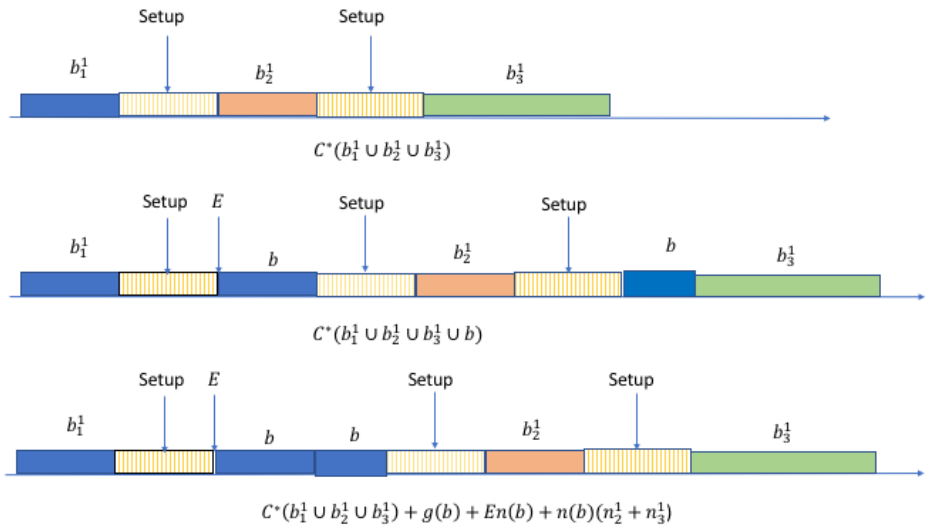

Figure A.2: Insert a Batch

Lemma 23. *Suppose I is a job instance, b is a batch and $p_{min} = 1$, then $C^*(I \cup b) \geq C^*(I) + g(b) + En(b) + n(b)(n(I) - q)$, where E is the time when the server starts serving batch b in the optimal solution, and q is the number of jobs in I that are served before time E.*

Proof. Suppose the truncated optimal solution is given for I. Now we consider the union instance $I \cup b$. If all jobs in b are of workload $p_{min} = 1$, then in the optimal solution if all the jobs in b are available at time E, then they are processed together. Since there is no idling time in the truncated optimal solution $C^*(I \cup b)$, we gather all the jobs of b to be processed together and start processing batch b from time E to get the lower bound for $C^*(I \cup b)$. Then jobs from I that are served after E (with total number $(n(I) - q)$) are moved after batch b, resulting a delay of $n(b)(n(I) - q)$ for these jobs. A example is given in Figure A.2. The first schedule in Figure A.2 is the truncated optimal for the batch $b_1^1 \cup b_2^1 \cup b_3^1$. The second schedule is the truncated optimal schedule for $b_1^1 \cup b_2^1 \cup b_3^1 \cup b$, where b is separated into two parts. If all jobs in b are of workload $p_{min} = 1$, it is always beneficial to schedule all jobs of b in the same batch, which is shown as the third schedule in Figure A.2. Notice in Figure A.2, $q = n(b_1^1) = n_1^1$. □

Fact 24. *For positive numbers $\{a_i, b_i\}_{i=1}^n$, we have $\frac{\sum_{i=1}^n a_i}{\sum_{i=1}^n b_i} \leq \max_{i=1}^n \{\frac{a_i}{b_i}\}$.*

Proof. Without loss of generality, assume $\frac{a_n}{b_n} = \max_{i=1}^n \{\frac{a_i}{b_i}\}$, then for any i we have $\frac{a_n}{b_n} \geq \frac{a_i}{b_i}$, thus $a_n b_i \geq a_i b_n$ holds for all $i = 1, ..., n$. Since $\frac{a_n}{b_n} - \frac{\sum_{i=1}^n a_i}{\sum_{i=1}^n b_i} = \frac{a_n \sum_{i=1}^n b_i - b_n \sum_{i=1}^n a_i}{b_n(\sum_{i=1}^n b_i)} = \frac{\sum_{i=1}^n (a_n b_i - a_i b_n)}{b_n(\sum_{i=1}^n b_i)} \geq 0$, we have $\frac{\sum_{i=1}^n a_i}{\sum_{i=1}^n b_i} \leq \frac{a_n}{b_n} = \max_{i=1}^n \{\frac{a_i}{b_i}\}$.

Next we restate Theorem 1 in the main paper and describe the proof. □

Theorem 25. ***(Theorem 1 in the main paper)*** *Any policy in Π_1 has competitive ratio of $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$ for the polling system $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$. This competitive ratio is tight when $\frac{3}{2}\gamma \leq k+1$.*

Proof. We prove the theorem by induction. We first show the result holds for $I^1 \cup b_1^2$, and then we show the result holds for $(\cup_{j=1}^{w-1} I^j) \cup (\cup_{i=1}^{l+1} b_i^w)$ if the result holds for $(\cup_{j=1}^{w-1} I^j) \cup (\cup_{i=1}^{l} b_i^w)$. We consider an instance

for which the cyclic policy works without idling, i.e., a busy period. If $p_{min} > 0$, we can assume an appropriate time unit so that $p_{min} = 1$ and $p_{max} = \gamma$ (we shall show later the case where $p_{min} = 0$). Notice that $I^1 = \cup_{i=1}^k b_i^1$ is the union of batches served in the first cycle under the online policy. First we show $C^e(I^1) \leq \kappa C^*(I^1)$ where $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$ for I-B and $C^e(I)$ is the completion time for instance I under a specific policy in Π_1. Without loss of generality, we assume the server serves from queue 1 to queue k in each cycle. Knowing that $I^1 = \cup_{i=1}^k b_i^1$, we let $n_{(k)}^1 \geq n_{(k-1)}^1 \geq ... \geq n_{(1)}^1$ be the descending permutation of $(n_1^1, ..., n_k^1)$, and $E^1 = \min_{i=1}^k E_i^1$, $S^1 = \min_{i=1}^k S_i^1$. Then we have (with explanation given later)

$$C^*(I^1) \quad \geq \quad g(I^1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n_{(k-j+1)}^1 + E^1 \sum_{i=1}^{k} n_i^1, \tag{A.1}$$

and

$$C^e(I^1) \quad \leq \quad \gamma g(I^1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n_{(j)}^1 + S^1 \sum_{i=1}^{k} n_i^1. \tag{A.2}$$

The RHS of Inequality (A.1) is the minimal completion time of a list which has the same number of jobs in each queue as I^1 and arrives at time E^1, and each job in this list is of processing time 1. The first term $g(I^1)$ is the pure completion time. The second term is because $n_{(k)}^1 \geq n_{(k-1)}^1 \geq ... \geq n_{(1)}^1$, if all batches are available at time E^1 and there is no further arrivals, the best order of serving the batches is to serve from the longest one to the shortest one. Note that the optimal policy may start without setting up since it may be the same queue that the server was idling at and resumed with. In the case where there is no setup for the first queue, we have the completion time due to setup to be $\tau \sum_{i=2}^k \sum_{j=i}^k n_{(k-j+1)}^1$. The third term in RHS of Inequality (A.1) is because the entire service process for the optimal solution starts from E^1. Therefore, the RHS of Inequality (A.1) is a lower bound for $C^*(I^1)$. The RHS of inequality (A.2) is the upper bound for the online policy, which says the online policy may serve batches from the shortest to the longest, starting from time point S^1, and all jobs are of the maximal workload γ. Since we consider I-B in this case, the server in the online policy does not set up for the first batch as the server was idling in the same queue as the new arrival. So the completion time resulted by setup is upper bounded by $\tau \sum_{i=2}^k \sum_{j=i}^k n_{(j)}^1$.

Now we show $\frac{C^e(I^1 \cup b_1^2)}{C^*(I^1 \cup b_1^2)} \leq \kappa$ for I-B. Before that, we let $Z(k) = \sum_{i=1}^k \sum_{j=i}^k n_{(j)}^1$ and $Z^*(k) = \sum_{i=1}^k \sum_{j=i}^k n_{(k-j+1)}^1$ then

$$\frac{Z(k)}{Z^*(k)} \quad = \quad \frac{kn_{(k)}^1 + (k-1)n_{(k-1)}^1 + ... + n_{(1)}^1}{kn_{(1)}^1 + (k-1)n_{(2)}^1 + ... + n_{(k)}^1} \leq \frac{kn_{(k)}^1 + kn_{(k-1)}^1 + ... + kn_{(1)}^1}{n_{(1)}^1 + n_{(2)}^1 + ... + n_{(k)}^1} \leq k.$$

From Fact 24 we have

$$\begin{aligned} \frac{C^e(I^1)}{C^*(I^1)} \quad &\leq \quad \frac{\gamma g(I^1) + \tau \sum_{i=2}^k \sum_{j=i}^k n_{(i)}^1 + S^1 \sum_{i=1}^k n_i^1}{g(I^1) + \tau \sum_{i=2}^k \sum_{j=i}^k n_{(k-j+1)}^1 + E^1 \sum_{i=1}^k n_i^1} \leq \frac{\gamma g(I^1) + \tau(Z(k) - \sum_{i=1}^k n_i^1) + E^1 \sum_{i=1}^k n_i^1}{g(I^1) + \tau(Z^*(k) - \sum_{i=1}^k n_i^1) + E^1 \sum_{i=1}^k n_i^1} \\ &\leq \quad \max\{\gamma, k\} < \kappa. \end{aligned} \tag{A.3}$$

The Inequality (A.3) follows from the fact that $\tau \le \min_{i=1}^{k} S_i^1 = \min_{i=1}^{k}\{R_i^1\} \le \min_{i=1}^{k} E_i^1$ because there must be a busy period happening before an I-B. Now we show that $C^e(I^1 \cup b_1^2) \le \kappa C^*(I^1 \cup b_1^2)$. Clearly if $n_1^2 = 0$ then the conclusion holds. Now suppose $n_1^2 \neq 0$, then we have

$$C^e(I^1 \cup b_1^2) \quad \le \quad C^e(I^1) + \gamma g(b_1^2) + n_1^2 \left(\gamma \sum_{i=1}^{k} n_i^1 + k\tau + S^1 \right), \tag{A.4}$$

and

$$C^*(I^1 \cup b_1^2) \quad \ge \quad C^*(I^1) + g(b_1^2) + E_1^2 n_1^2 + n_1^2 (\sum_{i=1}^{k} n_i^1 - q_1^2), \tag{A.5}$$

where

$$E_1^2 \ge \quad 2\tau + q_1^2, \qquad \text{if } q_1^2 \neq 0 \tag{A.6}$$

and

$$E_1^2 \ge R_1^2 > S_1^1 + n_1^1 \ge S^1 + n_1^1. \tag{A.7}$$

The RHS of Inequality (A.4) is because the completion time of batch b_1^2 is bounded by γ times its pure completion time $g(b_1^2)$, which is the maximal pure completion time for b_1^2 (if all the workload in b_1^2 is $p_{max} = \gamma$), plus n_1^2 times the maximal starting time $\gamma \sum_{i=1}^{k} n_i^1 + k\tau + S^1$. The RHS of Inequality (A.5) follows by a similar idea, and the last term of Inequality (A.5) is because q_1^2 is the number of jobs that have been served before batch b_1^2, so $\sum_{i=1}^{k} n_i^1 - q_1^2$ number of jobs will be served after b_1^2, which will incur at least $n_1^2(\sum_{i=1}^{k} n_i^1 - q_1^2)$ amount of completion time (following the idea of truncated optimal solution in lemma 23). The Inequality (A.6) holds because before E_1^2, the server has served at least one queue other than queue 1, thus there must be at least one setup before setting up for queue 1. If $q_1^2 \neq 0$, then following from a convex combination of Inequality (A.6) and (A.7) we have

$$E_1^2 \quad \ge \quad \frac{2}{3}(2\tau + q_1^2) + \frac{1}{3}(S^1 + n_1^1).$$

From $\kappa = \max\{\frac{3}{2}\gamma, k+1\} \ge \frac{3}{2}\gamma$ we have $\gamma \le \frac{2}{3}\kappa$. Then

$$\begin{aligned}
\kappa C^*(I^1 \cup b_1^2) - C^e(I^1 \cup b_1^2) \quad &\ge \quad \kappa C^*(I^1) - C^e(I^1) + (\kappa - \gamma) g(b_1^2) \\
&\quad + \kappa \left(E_1^2 n_1^2 + n_1^2 (\sum_{i=1}^{k} n_i^1 - q_1^2) \right) - n_1^2 \left(\gamma \sum_{i=1}^{k} n_i^1 + k\tau + S^1 \right) \\
&\ge \quad n_1^2 \left(\kappa (\frac{4}{3}\tau + \frac{2}{3} q_1^2 + \frac{1}{3} S^1 + \frac{1}{3} n_1^1 + \sum_{i=1}^{k} n_i^1 - q_1^2) - \left(\gamma \sum_{i=1}^{k} n_i^1 + k\tau + S^1 \right) \right)
\end{aligned}$$

$$\begin{aligned} &\geq \quad n_1^2\left(\frac{4}{3}\kappa - k\right)\tau + n_1^2\left(\frac{\kappa}{3} - 1\right)S^1 + n_1^2\left((\kappa-\gamma)\sum_{i=1}^{k} n_i^1 - \frac{\kappa}{3}q_1^2\right) \\ &\geq \quad 0. \end{aligned}$$

If $q_1^2 = 0$, then

$$\begin{aligned} \kappa C^*(I^1 \cup b_1^2) - C^e(I^1 \cup b_1^2) \quad &\geq \quad \kappa C^*(I^1) - C^e(I^1) + (\kappa-\gamma)g(b_1^2) + \kappa\left(E_1^2 n_1^2 + n_1^2\sum_{i=1}^{k} n_i^1\right) \\ &\quad -n_1^2\left(\gamma\sum_{i=1}^{k} n_i^1 + k\tau + S^1\right) \\ &\geq \quad n_1^2\left(\kappa(S^1 + n_1^1 + \sum_{i=1}^{k} n_i^1) - (\gamma\sum_{i=1}^{k} n_i^1 + k\tau + S^1)\right) \\ &\geq \quad n_1^2\left(\kappa S^1 - k\tau - S^1\right) \\ &\geq \quad n_1^2 k(S^1 - \tau) \\ &\geq \quad 0. \end{aligned} \tag{A.8}$$

The Inequality (A.8) follows from the fact that I^1 belongs to I-B so that $S^1 \geq \tau$. So we prove that $C^e(I^1 \cup b_1^2) \leq (k+1)C^*(I^1 \cup b_1^2)$ for I-B.

Now suppose the result holds for $\bar{b} = \left(\cup_{j=1}^{w-1} I^j\right) \cup \left(\cup_{i=1}^{l} b_i^w\right)$ where $w \geq 2$, and we want to show it also holds for $\bar{b} \cup b_{l+1}^w = \left(\cup_{j=1}^{w-1} I^j\right) \cup \left(\cup_{i=1}^{l+1} b_i^w\right)$ by induction, where $n_{l+1}^w \neq 0$. Let $m_l^w = \sum_{j=1}^{w-1}\sum_{i=1}^{k} 1_{n_i^j \neq 0} + \sum_{i=1}^{l} 1_{n_i^w \neq 0}$, be the number of non-empty batches the server has served before b_{l+1}^w in online policy, and $\bar{n} = \sum_{j=1}^{w-1}\sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l} n_i^w$ be the number of jobs served before b_{l+1}^w. Because the server stays in queue i at the k^{th} visit for no more than time γn_i^k and serves n_i^k jobs ,we have

$$C^e(\bar{b} \cup b_{l+1}^w) \quad \leq \quad C^e(\bar{b}) + \gamma g(b_{l+1}^w) + n_{l+1}^w\left(\gamma\bar{n} + (m_{l+1}^w - 1)\tau + S^1\right),$$

and

$$C^*(\bar{b} \cup b_{l+1}^w) \quad \geq \quad C^*(\bar{b}) + g(b_{l+1}^w) + E_{l+1}^w n_{l+1}^w + n_{l+1}^w(\bar{n} - q_{l+1}^w),$$

where

$$E_{l+1}^w \quad \geq \quad 2\tau + q_{l+1}^w \ \text{ if } q_{l+1}^w \neq 0,$$

and

$$E_{l+1}^w \quad \geq R_{l+1}^w > S_{l+1}^{w-1} + n_{l+1}^{w-1} > \quad \sum_{j=1}^{w-2}\sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + (m_{l+1}^{w-1} - 1)1_{m_{l+1}^{w-1} \geq 1}\tau + S^1.$$

Thus if $q_{l+1}^w = 0$ and $m_{l+1}^{w-1} \geq 2$

$$\begin{aligned}
\kappa C^*(\bar{b} \cup b_{l+1}^w) - C^e(\bar{b} \cup b_{l+1}^w) &\geq \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa - \gamma) g(b_{l+1}^w) \\
&+ \kappa n_{l+1}^w \left(\sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + (m_{l+1}^{w-1} - 1)\tau + S^1 + \bar{n} \right) \\
&\quad - n_{l+1}^w \left(\gamma \bar{n} + (m_{l+1}^w - 1)\tau + S^1 \right) \\
&\geq \kappa (\sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1}) n_{l+1}^w \\
&\quad + \kappa \left((m_{l+1}^{w-1} - 1)\tau + S^1 \right) n_{l+1}^w - ((m_{l+1}^w - 1)\tau + S^1) n_{l+1}^w \\
&\geq n_{l+1}^w \tau \left(\kappa m_{l+1}^{w-1} - \kappa - m_{l+1}^w + 1 \right) + n_{l+1}^w (\kappa - 1) S^1 \\
&\geq n_{l+1}^w \tau \left(\kappa m_{l+1}^{w-1} - \kappa - m_{l+1}^{w-1} - k + 1 \right) + n_{l+1}^w (\kappa - 1) S^1 \qquad \text{(A.9)} \\
&\geq n_{l+1}^w \tau \left(2(\kappa - 1) - \kappa - k + 1 \right) = n_{l+1}^w \tau (\kappa - k - 1) \geq 0.
\end{aligned}$$

Inequality (A.9) follows from the fact that $m_{l+1}^{w-1} - m_{l+1}^w \geq m_{l+1}^{w-1} - (m_{l+1}^{w-1} + k)$. If $q_{l+1}^w = 0$ and $0 \leq m_{l+1}^{w-1} \leq 1$, then $(m_{l+1}^{w-1} - 1) 1_{m_{l+1}^{w-1} \geq 1} \tau = 0$ and $m_{l+1}^w \leq k + 1$. Therefore

$$\begin{aligned}
\kappa C^*(\bar{b} \cup b_{l+1}^w) - C^e(\bar{b} \cup b_{l+1}^w) &\geq \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa - \gamma) g(b_{l+1}^w) \\
&+ \kappa \left(\left(\sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + S^1 \right) n_{l+1}^w + n_{l+1}^w \bar{n} \right) \\
&\quad - n_{l+1}^w \left(\gamma \bar{n} + (m_{l+1}^w - 1)\tau + S^1 \right) \\
&\geq \kappa (\sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1}) n_{l+1}^w + (\kappa - 1) S^1 n_{l+1}^w - (m_{l+1}^w - 1)\tau n_{l+1}^w \\
&\geq n_{l+1}^w \tau \left(\kappa - 1 - k \right) \qquad \text{(A.10)} \\
&\geq 0.
\end{aligned}$$

Inequality (A.10) follows from that fact that $S^1 \geq \tau$. If $q_{l+1}^w \neq 0$, we have

$$\begin{aligned}
E_{l+1}^w &\geq \frac{1}{3} \left(S^1 + (m_{l+1}^{w-1} - 1) 1_{m_{l+1}^{w-1} - 1 \geq 0} \tau + \sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} \right) + \frac{2}{3} \left(2\tau + q_{l+1}^w \right) \\
&= \frac{1}{3} (\sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1}) + \frac{1}{3} \left(S^1 + (m_{l+1}^{w-1} - 1) 1_{m_{l+1}^{w-1} - 1 \geq 0} \tau + 4\tau + 2q_{l+1}^w \right).
\end{aligned}$$

Thus if $q_{l+1}^w \neq 0$ and $m_{l+1}^{w-1} \geq 2$, then

$$\begin{aligned}
\kappa C^*(\bar{b} \cup b_{l+1}^w) - C^e(\bar{b} \cup b_{l+1}^w) &\geq \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa - \gamma) g(b_{l+1}^w) \\
&+ \kappa n_{l+1}^w \left(\frac{1}{3} (\sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1}) + \frac{1}{3} \left(S^1 + (m_{l+1}^{w-1} - 1)\tau + 4\tau + 2q_{l+1}^w \right) \right)
\end{aligned}$$

$$
\begin{aligned}
&+ \quad \kappa n_{l+1}^w(\bar{n} - q_{l+1}^w) - n_{l+1}^w(\gamma\bar{n} + (m_{l+1}^w - 1)\tau + S^1) \\
&\geq \quad n_{l+1}^w\tau(\frac{4}{3}\kappa + \frac{\kappa}{3}m_{l+1}^{w-1} - \frac{\kappa}{3} - m_{l+1}^{w-1} - k + 1) + n_{l+1}^w(\kappa\bar{n} - \frac{\kappa}{3}q_{l+1}^w - \gamma\bar{n}) \\
&\quad +n_{l+1}^w S^1(\frac{\kappa}{3} - 1) \\
&\geq \quad n_{l+1}^w\tau(\frac{\kappa - 3}{3}m_{l+1}^{w-1} + \kappa - k + 1) + n_{l+1}^w\frac{\kappa}{3}(\bar{n} - q_{l+1}^w) \\
&\geq \quad n_{l+1}^w\tau(k + 1 - k + 1) + n_{l+1}^w\frac{\kappa}{3}(\bar{n} - q_{l+1}^w) \geq 0.
\end{aligned}
$$

Then if $0 \leq m_{l+1}^{w-1} \leq 1$, we have

$$
\begin{aligned}
\kappa C^*(\bar{b} \cup b_{l+1}^w) - C^e(\bar{b} \cup b_{l+1}^w) \quad &\geq \quad \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa - \gamma)g(b_{l+1}^w) \\
&+ \quad \kappa n_{l+1}^w \left(\sum_{j=1}^{w-2}\sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + \frac{1}{3}\left(S^1 + 4\tau + 2q_{l+1}^w\right) \right) \\
&+ \quad \kappa n_{l+1}^w(\bar{n} - q_{l+1}^w) - n_{l+1}^w(\gamma\bar{n} + k\tau + S^1) \\
&\geq \quad n_{l+1}^w\tau(\frac{4}{3}\kappa - k) + n_{l+1}^w(\kappa\bar{n} - \frac{\kappa}{3}q_{l+1}^w - \gamma\bar{n}) + n_{l+1}^w S^1(\frac{\kappa}{3} - 1) \\
&\geq \quad n_{l+1}^w\tau(\frac{4}{3}\kappa - k) + n_{l+1}^w\frac{\kappa}{3}(\bar{n} - q_{l+1}^w) \geq 0.
\end{aligned}
$$

Now we show that results hold for the second type of busy period, i.e., II-B. For II-B, the first arrival occurs in a different queue from where the server was idling, thus the server starts this period with a setup. Note that the very first busy period is also a II-B. If I^1 belongs to the first busy period, then

$$
C^*(I^1) \quad \geq \quad g(I^1) + \tau\sum_{i=1}^{k}\sum_{j=i}^{k} n_{(k-j+1)}^1.
$$

$$
C^e(I^1) \quad \leq \quad \gamma g(I^1) + \tau\sum_{i=1}^{k}\sum_{j=i}^{k} n_{(j)}^1.
$$

Thus

$$
\frac{C^e(I^1)}{C^*(I^1)} \quad \leq \quad \frac{\gamma g(I^1) + Z(k)}{g(I^1) + Z^*(k)} \leq \max\{\gamma, k\} \leq \kappa.
$$

If I^1 does not belong to the first busy period, then

$$
C^*(I^1) \quad \geq \quad g(I^1) + \tau\sum_{i=2}^{k}\sum_{j=i}^{k} n_{(k-j+1)}^1 + E^1\sum_{i=1}^{k} n_i^1,
$$

and

$$
C^e(I^1) \quad \leq \quad \gamma g(I^1) + \tau\sum_{i=1}^{k}\sum_{j=i}^{k} n_{(j)}^1 + S^1\sum_{i=1}^{k} n_i^1.
$$

Thus if $R^1 \geq 2\tau$, then $\frac{R^1+\tau}{R^1-\tau} \leq 3$, then

$$\begin{aligned}
\frac{C^e(I^1)}{C^*(I^1)} &\leq \frac{\gamma g(I^1) + \tau \sum_{i=1}^k \sum_{j=i}^k n^1_{(j)} + S^1 \sum_{i=1}^k n_i^1}{g(I^1) + \tau \sum_{i=2}^k \sum_{j=i}^k n^1_{(k-j+1)} + E^1 \sum_{i=1}^k n_i^1} \\
&\leq \frac{\gamma g(I^1) + \tau \sum_{i=1}^k \sum_{j=i}^k n^1_{(j)} + (R^1+\tau) \sum_{i=1}^k n_i^1}{g(I^1) + \tau \sum_{i=2}^k \sum_{j=i}^k n^1_{(k-j+1)} + R^1 \sum_{i=1}^k n_i^1} \\
&\leq \frac{\gamma g(I^1) + \tau Z(k) + (R^1+\tau) \sum_{i=1}^k n_i^1}{g(I^1) + \tau Z^*(k) + (R^1-\tau) \sum_{i=1}^k n_i^1} \qquad \text{(A.11)} \\
&\leq \max\{\gamma, k, 3\} \\
&< \kappa.
\end{aligned}$$

Inequality (A.11) follows from $E^1 \geq R^1 = \min_{i=1}^k R_i^1 \geq \tau$ and $S^1 = R^1 + \tau$ because the server would immediately set up the queue where a new arrival occurs after an idling period. If $R^1 < 2\tau$, then the online policy has only scheduled at most one batch before R^1. Then the online policy and the optimal policy should start from the same queue at II-B, in which $R^1 = E^1 = S^1$ and

$$\begin{aligned}
\frac{C^e(I^1)}{C^*(I^1)} &\leq \frac{\gamma g(I^1) + \tau \sum_{i=1}^k \sum_{j=i}^k n^1_{(j)} + S^1 \sum_{i=1}^k n_i^1}{g(I^1) + \tau \sum_{i=1}^k \sum_{j=i}^k n^1_{(k-j+1)} + S^1 \sum_{i=1}^k n_i^1} \\
&\leq \max\{\gamma, k, 1\} < \kappa.
\end{aligned}$$

We have

$$C^e(I^1 \cup b_1^2) \leq C^e(I^1) + \gamma g(b_1^2) + n_1^2 \left(\gamma \sum_{i=1}^k n_i^1 + (k+1)\tau + R^1 \right),$$

and

$$C^*(I^1 \cup b_1^2) \geq C^*(I^1) + g(b_1^2) + E_1^2 n_1^2 + n_1^2 (\sum_{i=1}^k n_i^1 - q_1^2),$$

where

$$E_1^2 \geq 2\tau + q_1^2, \quad \text{if } q_1^2 \neq 0$$

and

$$E_1^2 \geq R_1^2 > S_1^1 + n_1^1 \geq S^1 + n_1^1 = R^1 + \tau + n_1^1.$$

If $q_1^2 = 0$, then

$$\begin{aligned}\kappa C^*(I^1\cup b_1^2)-C^e(I^1\cup b_1^2) &\geq \kappa C^*(I^1)-C^e(I^1)+(\kappa-\gamma)g(b_1^2)\\ &\quad +\kappa\left(E_1^2n_1^2+n_1^2\sum_{i=1}^k n_i^1\right)-n_1^2\left(\gamma\sum_{i=1}^k n_i^1+(k+1)\tau+R^1\right)\\ &\geq n_1^2\left(\kappa(R^1+\tau+n_1^1+\sum_{i=1}^k n_i^1)-\left(\gamma\sum_{i=1}^k n_i^1+(k+1)\tau+R^1\right)\right)\\ &\geq n_1^2\tau\left(\kappa-(k+1)\right)\geq 0.\end{aligned}$$

If $q_1^2\neq 0$, then

$$E_1^2 \geq \frac{2}{3}(2\tau+q_1^2)+\frac{1}{3}(R^1+\tau+n_1^1)=\frac{5}{3}\tau+\frac{1}{3}n_1^1+\frac{1}{3}R^1+\frac{2}{3}q_1^2.$$

Thus

$$\begin{aligned}\kappa C^*(I^1\cup b_1^2)-C^e(I^1\cup b_1^2) &\geq \kappa C^*(I^1)-C^e(I^1)+(\kappa-\gamma)g(b_1^2)\\ &\quad \kappa\left(E_1^2n_1^2+n_1^2(\sum_{i=1}^k n_i^1-q_1^2)\right)-n_1^2\left(\gamma\sum_{i=1}^k n_i^1+(k+1)\tau+R^1\right)\\ &\geq n_1^2\left(\frac{5}{3}\kappa\tau+\frac{\kappa}{3}n_1^1+\frac{\kappa}{3}R^1+\kappa\sum_{i=1}^k n_i^1-\frac{\kappa}{3}q_1^2-\left(\gamma\sum_{i=1}^k n_i^1+(k+1)\tau+R^1\right)\right)\\ &\geq n_1^2\left((\frac{5}{3}\kappa-k-1)\tau+(\kappa-\gamma)\sum_{i=1}^k n_i^1-\frac{\kappa}{3}q_1^2+(\frac{\kappa}{3}-1)R^1\right)\geq 0.\end{aligned}$$

Now we suppose the result holds for $\bar{b}=\left(\cup_{j=1}^{w-1}I^j\right)\cup\left(\cup_{i=1}^l b_i^w\right)$ where $w\geq 2$, now we want to see it also holds for $\bar{b}\cup b_{l+1}^w$ and $n_{l+1}^w\neq 0$. Let $m_l^w=\sum_{j=1}^{w-1}\sum_{i=1}^k 1_{n_i^j\neq 0}+\sum_{i=1}^l 1_{n_i^w\neq 0}$ and $\bar{n}=\sum_{j=1}^{w-1}\sum_{i=1}^k n_i^j+\sum_{i=1}^l n_i^w$. Since the server stays in queue i at the k^{th} visit for no more than γn_i^k and serves at least n_i^k jobs, we have

$$C^e(\bar{b}\cup b_{l+1}^w) \leq C^e(\bar{b})+\gamma g(b_{l+1}^w)+n_{l+1}^w\left(\gamma\bar{n}+m_{l+1}^w\tau+R^1\right)$$

and

$$C^*(\bar{b}\cup b_{l+1}^w) \geq C^*(\bar{b})+g(b_{l+1}^w)+E_{l+1}^w n_{l+1}^w+n_{l+1}^w(\bar{n}-q_{l+1}^w)$$

where

$$E_{l+1}^w \geq 2\tau+q_{l+1}^w, \text{ if } q_{l+1}^w\neq 0$$

and

$$E^w_{l+1} \quad \geq R^w_{l+1} > S^{w-1}_{l+1} + n^{w-1}_{l+1} > \quad \sum_{j=1}^{w-2}\sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i + m^{w-1}_{l+1}\tau + R^1.$$

If $q^w_{l+1} = 0$ and $m^{w-1}_{l+1} \geq 1$, then by a similar argument as the case I-B we have

$$\begin{aligned}
\kappa C^*(\bar{b} \cup b^w_{l+1}) - C^e(\bar{b} \cup b^w_{l+1}) &\geq \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa - \gamma) g(b^w_{l+1}) \\
&+ \kappa n^w_{l+1} \left(\sum_{j=1}^{w-2}\sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i + m^{w-1}_{l+1}\tau + R^1 + \bar{n} \right) \\
&\quad -n^w_{l+1} \left(\gamma\bar{n} + m^w_{l+1}\tau + R^1 \right) \\
&\geq \kappa(\sum_{j=1}^{w-2}\sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i) n^w_{l+1} + \kappa \left(m^{w-1}_{l+1}\tau + R^1 \right) n^w_{l+1} \\
&\quad -(m^w_{l+1}\tau + R^1) n^w_{l+1} \\
&\geq n^w_{l+1}\tau \left(\kappa m^{w-1}_{l+1} - m^w_{l+1} \right) + n^w_{l+1}(\kappa - 1) R^1 \\
&\geq n^w_{l+1}\tau \left(\kappa m^{w-1}_{l+1} - m^{w-1}_{l+1} - k \right) + n^w_{l+1}(\kappa - 1) R^1 \\
&\geq n^w_{l+1}\tau \left(\kappa - 1 - k \right) \geq 0.
\end{aligned}$$

If $q^w_{l+1} = 0$, $m^{w-1}_{l+1} = 0$ and $R^1 \geq \tau$, then

$$\begin{aligned}
\kappa C^*(\bar{b} \cup b^w_{l+1}) - C^e(\bar{b} \cup b^w_{l+1}) &\geq \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa - \gamma) g(b^w_{l+1}) \\
&+ \kappa n^w_{l+1} \left(\sum_{j=1}^{w-2}\sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i + R^1 + \bar{n} \right) - n^w_{l+1} \left(\gamma\bar{n} + k\tau + R^1 \right) \\
&\geq \kappa(\sum_{j=1}^{w-2}\sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i) n^w_{l+1} + \kappa R^1 n^w_{l+1} - (k\tau + R^1) n^w_{l+1} \\
&\geq -n^w_{l+1} k\tau + n^w_{l+1}(\kappa - 1) R^1 \\
&\geq n^w_{l+1}\tau \left(\kappa - 1 - k \right) \geq 0.
\end{aligned}$$

If $q^w_{l+1} = 0$, $m^{w-1}_{l+1} = 0$ and $R^1 < \tau$, since $E^w_{l+1} \geq E^1 \geq \tau$, then

$$\begin{aligned}
\kappa C^*(\bar{b} \cup b^w_{l+1}) - C^e(\bar{b} \cup b^w_{l+1}) &\geq \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa - \gamma) g(b^w_{l+1}) + \kappa n^w_{l+1} (\tau + \bar{n}) - n^w_{l+1} \left(\gamma\bar{n} + m^w_{l+1}\tau + \tau \right) \\
&\geq n^2_{l+1}\tau \left(\kappa - k - 1 \right) + n^2_{l+1}(\kappa - \gamma)\bar{n} \geq 0.
\end{aligned}$$

If $q^w_{l+1} \neq 0$, from the fact that

$$
\begin{aligned}
E^w_{l+1} &\geq \frac{1}{3}\left(R^1 + m^{w-1}_{l+1}\tau + \sum_{j=1}^{w-2}\sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i\right) + \frac{2}{3}\left(2\tau + q^w_{l+1}\right) \\
&= \frac{1}{3}R^1 + \frac{1}{3}(m^{w-1}_{l+1} + 4)\tau + \frac{2}{3}q^w_{l+1} + \frac{1}{3}(\sum_{j=1}^{w-2}\sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i),
\end{aligned}
$$

we have

$$
\begin{aligned}
\kappa C^*(\bar{b}\cup b^w_{l+1}) - C^e(\bar{b}\cup b^w_{l+1}) &\geq \kappa C^*(\bar{b}) - C^e(\bar{b}) + (\kappa-\gamma)g(b^w_{l+1}) \\
&+ \kappa n^w_{l+1}\left(\frac{1}{3}R^1 + \frac{1}{3}m^{w-1}_{l+1}\tau + \frac{4}{3}\tau - \frac{1}{3}q^w_{l+1} + \frac{1}{3}(\sum_{j=1}^{w-2}\sum_{i=1}^{l} n^j_i + \sum_{i=1}^{l+1} n^{w-1}_i) + \bar{n}\right) \\
&- n^w_{l+1}(\gamma\bar{n} + m^w_{l+1}\tau + R^1) \\
&\geq n^w_{l+1}\left((\kappa-\gamma)\bar{n} - \frac{\kappa}{3}q^w_{l+1}\right) + n^w_{l+1}\tau(\frac{\kappa}{3}m^{w-1}_{l+1} + \frac{4}{3}\kappa - m^{w-1}_{l+1} - k) + n^w_{l+1}R^1(\frac{\kappa}{3} - 1) \\
&\geq n^w_{l+1}\tau(\frac{\kappa-3}{3}m^{w-1}_{l+1} + \frac{4\kappa}{3} - k) > 0.
\end{aligned}
$$

Hence we prove the results hold for II-B. If $p_{max} = 0$, then $p_{min} = 0$, then we have for I-B that

$$
C^*(I^1) \geq \tau\sum_{i=2}^{k}\sum_{j=i}^{k} n^1_{(k-j+1)} + E^1\sum_{i=1}^{k} n^1_i,
$$

and

$$
C^e(I^1) \leq \tau\sum_{i=2}^{k}\sum_{j=i}^{k} n^1_{(i)} + S^1\sum_{i=1}^{k} n^1_i.
$$

By a similar argument as before we have $\frac{C^*(I^1)}{C^e(I^1)} \leq \kappa$. Likewise, we have

$$
C^e(I^1\cup b^2_1) \leq C^e(I^1) + n^2_1\left(k\tau + S^1\right)
$$

and

$$
C^*(I^1\cup b^2_1) \geq C^*(I^1) + E^2_1 n^2_1,
$$

where

$$
E^2_1 \geq 2\tau, \quad \text{if } q^2_1 \neq 0
$$

and

$$E_1^2 \geq R_1^2 > S^1 + n_1^1.$$

We also have if $q_1^2 = 0$, then

$$\begin{aligned} \kappa C^*(I^1 \cup b_1^2) - C^e(I^1 \cup b_1^2) &\geq \kappa C^*(I^1) - C^e(I^1) + \kappa n_1^2(S^1 + n_1^1) - n_1^2(k\tau + S^1) \\ &\geq n_1^2(kS^1 - k\tau) \geq 0. \end{aligned}$$

Similarly, we can show all the other arguments hold if $p_{min} = p_{max} = 0$.

To show the competitive ratio is tight when $\gamma \leq \frac{3}{2}(k+1)$, we show a special instance I. Suppose there are k queues in the system and at time 0 there is one job with workload γ at each queue. At time $\gamma + \tau + \epsilon$ there is a batch b_1^2 of n_1^2 jobs each of workload γ that arrive at queue 1. We thus have

$$C^e(I) = \gamma g(I^1 \cup b_1^2) + (n_1^2 + k)\tau + (n_1^2 + k - 1)\tau + ... + n_1^2\tau,$$

and

$$C^*(I) = \gamma g(I^1 \cup b_1^2) + (n_1^2 + k)\tau + (n_1^2 + k - 1)\epsilon + (k-1)\tau + ... + \tau.$$

Thus

$$\frac{C^e(I)}{C^*(I)} = 1 + k$$

as $\tau > (n_1^2)^2$ and $n \to \infty$. The result also holds for $p_{min} = 0$. □

B Proof for Theorem 2 in the Main Paper

Theorem 26. ***(Theorem 2 in the main paper)*** *Any policy in* Π_2 *has competitive ratio of* $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$ *for the polling system* $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$. *This competitive ratio is tight when* $\frac{3}{2}\gamma \leq k+1$.

Proof. The proof is similar to the one for Cyclic with Skipping the Empty Queue, however this time we only need to show $C^{ew}(I) \leq \kappa C^*(I)$ for any busy period I where $C^{ew}(I)$ is the completion time by a policy from Π_2. Notice we no longer need to consider different cases for I-B and II-B because even when the system is idling, the cyclic policy still keeps setting up queues in cycle. When a new arrival occurs after the system being empty for some time, we simply regard this time R^1 as the beginning of a busy period. Without loss of generality, we assume the server begins serving with queue 1 in this busy period. Let $n_{(k)}^1 \geq n_{(k-1)}^1 \geq ... \geq n_{(1)}^1$ be the descending permutation of $(n_1^1, ..., n_k^1)$, $R^1 = \min_{i=1}^k R_i^1$, $E^1 = \min_{i=1}^k E_i^1$ and $S^1 = \min_{i=1}^k S_i^1$. Notice some of $n_{(i)}^1$ may be zero (not all of them) but the server still sets up the queue even if the queue is empty. We have

$$C^*(I_1) \quad \geq \quad g(I_1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)} + E^1 \sum_{i=1}^{k} n^1_i,$$

and

$$C^{ew}(I_1) \quad \leq \quad \gamma g(I_1) + \tau k \sum_{i=1}^{k} n^1_{(i)} + R^1 \sum_{i=1}^{k} n^1_i.$$

To show $\frac{C^{ew}(I_1)}{C^*(I_1)} \leq \kappa$, by abusing the notation a little, we first let $Z(k) = k \sum_{i=1}^{k} n^1_{(i)}$ and $Z^*(k) = \sum_{i=1}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)}$, so

$$\frac{Z(k)}{Z^*(k)} = \frac{kn^1_{(k)} + kn^1_{(k-1)} + ... + kn^1_{(1)}}{kn^1_{(1)} + (k-1)n^1_{(2)} + ... + n^1_{(k)}} \leq \frac{kn^1_{(k)} + kn^1_{(k-1)} + ... + kn^1_{(1)}}{n^1_{(1)} + n^1_{(2)} + ... + n^1_{(k)}} \leq k.$$

Thus

$$\begin{aligned}
\frac{C^{ew}(I_1)}{C^*(I_1)} &\leq \frac{\gamma g(I_1) + \tau k \sum_{i=1}^{k} n^1_{(i)} + R^1 \sum_{i=1}^{k} n^1_i}{g(I_1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)} + E^1 \sum_{i=1}^{k} n^1_i} \\
&= \frac{\gamma g(I_1) + \tau k \sum_{i=1}^{k} n^1_{(i)} + R^1 \sum_{i=1}^{k} n^1_i}{g(I_1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)} + \max\{\tau, R^1\} \sum_{i=1}^{k} n^1_i} \\
&\leq \max\{\gamma, k+1\} \qquad \text{(B.1)} \\
&\leq \kappa.
\end{aligned}$$

Inequality (B.1) holds because if $R^1 \leq \tau$, then

$$\begin{aligned}
\frac{\gamma g(I_1) + \tau k \sum_{i=1}^{k} n^1_{(i)} + R^1 \sum_{i=1}^{k} n^1_i}{g(I_1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)} + E^1 \sum_{i=1}^{k} n^1_i} &\leq \frac{\gamma g(I_1) + \tau k \sum_{i=1}^{k} n^1_{(i)} + \tau \sum_{i=1}^{k} n^1_i}{g(I_1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)} + \tau \sum_{i=1}^{k} n^1_i} \\
&\leq \max\{\gamma, k+1\}.
\end{aligned}$$

And if $R^1 > \tau$, then

$$\begin{aligned}
\frac{\gamma g(I_1) + \tau k \sum_{i=1}^{k} n^1_{(i)} + R^1 \sum_{i=1}^{k} n^1_i}{g(I_1) + \tau \sum_{i=2}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)} + E^1 \sum_{i=1}^{k} n^1_i} &\leq \frac{\gamma g(I_1) + \tau(k+1) \sum_{i=1}^{k} n^1_{(i)} + (R^1 - \tau) \sum_{i=1}^{k} n^1_i}{g(I_1) + \tau \sum_{i=1}^{k} \sum_{j=i}^{k} n^1_{(k-j+1)} + (R^1 - \tau) \sum_{i=1}^{k} n^1_i} \\
&\leq \max\{\gamma, k+1, 1\}.
\end{aligned}$$

Now we show that $C^{ew}(I_1 \cup b^2_1) \leq \kappa C^*(I_1 \cup b^2_1)$. We thus have for non-empty batch b^2_1,

$$C^{ew}(I_1 \cup b^2_1) \quad \leq \quad C^{ew}(I_1) + \gamma g(b^2_1) + n^2_1 \left(\gamma \sum_{i=1}^{k} n^1_i + k\tau + S^1 \right),$$

and

$$C^*(I_1 \cup b_1^2) \quad \geq \quad C^*(I_1) + g(b_1^2) + E_1^2 n_1^2 + n_1^2(\sum_{i=1}^{k} n_i^1 - q_1^2),$$

where

$$E_1^2 \geq \quad 2\tau + q_1^2, \qquad \text{if } q_1^2 \neq 0$$

and

$$E_1^2 \geq R_1^2 > S^1 + n_1^1.$$

If $n_1^2 \neq 0$, following the same argument in Theorem 1 we have $C^{ew}(I_1 \cup b_1^2) \leq \kappa C(I_1 \cup b_1^2)$.

Now we suppose the result holds for $\cup_{j=1}^{w-1} I^j \cup (\cup_{i=1}^{l} b_i^w)$ where $w \geq 2$, if $n_{l+1}^w \neq 0$, then by similar argument in Theorem 1 we can show the results. We again abuse notation by letting $m_l^w = k(w-1) + l$, $\bar{b} = \cup_{j=1}^{w-1} I^j \cup (\cup_{i=1}^{l} b_i^w)$ and $\bar{n} = \sum_{j=1}^{w-1} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l} n_i^w$. We have

$$C^{ew}(\bar{b} \cup b_{l+1}^w) \quad \leq \quad C^{ew}(\bar{b}) + \gamma g(b_{l+1}^w) + n_{l+1}^w \left(\gamma \bar{n} + (m_{l+1}^w - 1)\tau + S^1\right),$$

and

$$C^*(\bar{b} \cup b_{l+1}^w) \quad \geq \quad C^*(\bar{b}) + g(b_{l+1}^w) + E_{l+1}^w n_{l+1}^w + n_{l+1}^w(\bar{n} - q_{l+1}^w),$$

where

$$E_{l+1}^w \quad \geq \quad 2\tau + q_{l+1}^w, \quad \text{if } q_{l+1}^w \neq 0$$

and

$$E_{l+1}^w \geq R_{l+1}^w > S_{l+1}^{w-1} + n_{l+1}^{w-1} > S^1 + \sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + (m_{l+1}^{w-1} - 1)\tau.$$

Note $m_{l+1}^{w-1} \geq 1$ since we suppose $w \geq 2$. Thus if $q_{l+1}^w = 0$, then

$$\begin{aligned} \kappa C^*(\bar{b} \cup b_{l+1}^w) - C^{ew}(\bar{b} \cup b_{l+1}^w) \quad &\geq \quad \kappa C^*(\bar{b}) - C^{ew}(\bar{b}) + (\kappa - \gamma) g(b_{l+1}^w) \\ &+ \quad \kappa \left(S^1 + \left(\sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + (m_{l+1}^{w-1} - 1)\tau \right) n_{l+1}^w + n_{l+1}^w(\bar{n} - q_{l+1}^w) \right) \\ &\quad -n_{l+1}^w \left(S^1 + \gamma \bar{n} + (m_{l+1}^w - 1)\tau\right) \end{aligned}$$

$$
\begin{aligned}
&\geq \kappa(\sum_{j=1}^{w-2}\sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1})n_{l+1}^w + n_{l+1}^w\tau(\kappa m_{l+1}^{w-1} - \kappa - m_{l+1}^{w-1} - k + 1) \\
&\quad +n_{l+1}^w(\kappa-1)S^1 \\
&\geq n_{l+1}^w\tau\left(\kappa - \kappa - 1 - k + 1 + \kappa - 1\right) \geq n_{l+1}^w\tau\left(\kappa - k - 1\right) \geq 0.
\end{aligned}
$$

If $q_{l+1}^w \neq 0$, then from the fact that

$$
\begin{aligned}
E_{l+1}^w &\geq \frac{1}{3}\left(S^1 + \sum_{j=1}^{w-2}\sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + (m_{l+1}^{w-1} - 1)\tau\right) + \frac{2}{3}(2\tau + q_{l+1}^w) \\
&= \frac{1}{3}\left(S^1 + \sum_{j=1}^{w-2}\sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + m_{l+1}^{w-1} + 3\tau + 2q_{l+1}^w\right),
\end{aligned}
$$

then

$$
\begin{aligned}
\kappa C^*(\bar{b}\cup b_{l+1}^w) - C^{ew}(\bar{b}\cup b_{l+1}^w) &\geq \kappa C^*(\bar{b}) - C^{ew}(\bar{b}) + (\kappa-\gamma)g(b_{l+1}^w) \\
&+ \kappa n_{l+1}^w\left(\frac{1}{3}\left(S^1 + \sum_{j=1}^{w-2}\sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l+1} n_i^{w-1} + m_{l+1}^{w-1} + 3\tau + 2q_{l+1}^w\right) + (\bar{n} - q_{l+1}^w)\right) \\
&\quad -n_{l+1}^w\left(S^1 + \gamma\bar{n} + (m_{l+1}^w - 1)\tau\right) \\
&\geq n_{l+1}^w(\frac{1}{3}\kappa - 1)S^1 + n_{l+1}^w\tau(\frac{\kappa}{3}m_{l+1}^{w-1} + \kappa - m_{l+1}^{w-1} - k + 1) + n_{l+1}^w(\kappa\bar{n} - \gamma\bar{n} - \frac{\kappa}{3}q_{l+1}^w) \\
&\geq n_{l+1}^w\tau\left(\kappa - k + 1\right) > 0.
\end{aligned}
$$

The case of $p_{min} = p_{max} = 0$ and the tightness of the competitive ratio are similar to the arguments in Theorem 1. □

C Proof for Theorem 3 in the Main Paper

Theorem 27. *(**Theorem 3 in the Main Paper**) Any policy in Π_3 has competitive ratio of $\kappa = \max\{\frac{3}{2}\gamma, k+1\}$ for the polling system $1 \mid r_i, \tau, p_{max} \leq \gamma p_{min} \mid \sum C_i$. This competitive ratio is tight when $\frac{3}{2}\gamma \leq k+1$.*

Proof. We only show the proof for the policy without skipping the empty queue. The proof is similar to the proof for Theorem 2. We prove the theorem by induction. Again for simplicity, we assume $p_{min} = 1$ so that $p_{max} = \gamma$. We want to show $C^g(I) \leq \kappa C^*(I)$ holds for any instance I, where $C^g(I)$ is the completion time for any policy $G \in \Pi_3$. Again we assume the server routes from queue 1 to queue k in each cycle. Let $n_{(k)}^1 \geq n_{(k-1)}^1 \geq ... \geq n_{(1)}^1$ is the descending permutation of $(n_1^1, ..., n_k^1)$ and $E^1 = \min_{i=1}^k E_i^1$, $S^1 = \min_{i=1}^k S_i^1$, we have

$$
C^*(I_1) \geq g(I_1) + \tau\sum_{i=2}^{k}\sum_{j=i}^{k} n_{(k-j+1)}^1 + E^1\sum_{i=1}^{k} n_i^1,
$$

and

$$C^g(I_1) \quad \leq \quad g(I_1) + \tau k \sum_{i=1}^{k} n^1_{(i)} + S^1 \sum_{i=1}^{k} n^1_i.$$

The proof of $\frac{C^g(I_1)}{C^*(I_1)} \leq \kappa$, is the same as in the proof for Theorem 2. Now we show that $C^g(I_1 \cup b^2_1) \leq \kappa C^*(I_1 \cup b^2_1)$. For gated policy, we know that $R^2_1 \geq S^1_1$.We then have

$$C^g(I_1 \cup b^2_1) \quad \leq \quad C^g(I_1) + \gamma g(b^2_1) + n^2_1 \left(\gamma \sum_{i=1}^{k} n^1_i + k\tau + S^1 \right),$$

and

$$C^*(I_1 \cup b^2_1) \quad \geq \quad C^*(I_1) + g(b^2_1) + E^2_1 n^2_1 + n^2_1 (\sum_{i=1}^{k} n^1_i - q^2_1),$$

where

$$E^2_1 \geq \quad 2\tau + q^2_1, \qquad \text{if } q^2_1 \neq 0$$

and

$$E^2_1 \geq R^2_1 > S^1 \geq \tau.$$

Since $n^2_1 \neq 0$, following the same argument in Theorem 2 we have $C^g(I_1 \cup b^2_1) \leq \kappa C^*(I_1 \cup b^2_1)$. Now we suppose the result holds for $(\cup_{j=1}^{w-1} I^j) \cup (\cup_{i=1}^{l} b^w_i)$ where $w \geq 2$, if $n^w_{l+1} \neq 0$, then by similar argument in Theorem 2 we can show the results. We let $m^w_l = k(w-1)+l$, $\bar{b} = (\cup_{j=1}^{w-1} I^j) \cup (\cup_{i=1}^{l} b^w_i)$ and $\bar{n} = \sum_{j=1}^{w-1} \sum_{i=1}^{k} n^j_i + \sum_{i=1}^{l} n^w_i$. We have

$$C^g(\bar{b} \cup b^w_{l+1}) \quad \leq \quad C^g(\bar{b}) + \gamma g(b^w_{l+1}) + n^w_{l+1} \left(\gamma \bar{n} + (m^w_{l+1} - 1)\tau + S^1 \right),$$

and

$$C^*(\bar{b} \cup b^w_{l+1}) \quad \geq \quad C^*(\bar{b}) + g(b^w_{l+1}) + E^w_{l+1} n^w_{l+1} + n^w_{l+1} (\bar{n} - q^w_{l+1}),$$

where

$$E^w_{l+1} \quad \geq \quad 2\tau + q^w_{l+1} \quad \text{if } q^w_{l+1} \neq 0$$

and

$$E^w_{l+1} \geq R^w_{l+1} > S^{w-1}_{l+1} > S^1 + \sum_{j=1}^{w-2} \sum_{i=1}^{k} n_i^j + \sum_{i=1}^{l} n_i^{w-1} + (m^{w-1}_{l+1} - 1)\tau.$$

By the similar argument in Theorem 2 we have the results hold. □

Notation	Meaning	Notation	Meaning
b_i^w, $i = 1, ..., k$	The job instance that are served by the online policy (cyclic based) during the w^{th} visit (w^{th} cycle) to queue i	I^w, $w = 1, 2, ...$	$I^w = \cup_{i=1}^{k} b_i^w$, the union of instances that are served by the online policy during the w^{th} cycle
$n_i^w = n(b_i^w)$, $i = 1, ..., k$	The number of jobs in batch b_i^w	$C^\pi(I)$	The total completion time of instance I by policy π
S_i^w, $i = 1, ..., k$	The time when the online policy starts to serve batch b_i^w	$S^1 = \min_{i=1}^{k}\{S_i^1\}$	The earliest staring time for I^1 by the online policy
R_i^w, $i = 1, ..., k$	The earliest release time (arrival time) of batch b_i^w	$R^1 = \min_{i=1}^{k}\{R_i^1\}$	The earliest release time of I^1
E_i^w	The time when the optimal offline policy starts to serve batch b_i^w	$E^1 = \min_{i=1}^{k}\{E_i^1\}$	The earliest time when the optimal offline policy starts to serve I^1
q_i^w, $i = 1...k$	The jobs in $(\cup_{j=1}^{w-1} I^j) \cup (\cup_{l=1}^{i-1} b_l^w)$ that have been served by the optimal policy before E_i^w	τ	Setup time
$g(I) = \frac{n(I)(n(I)+1)}{2}$	Pure completion time for instance I	Busy period	The time period between two consecutive empty periods
I-B	Type I busy period. The server start the new busy period without setting up	II-B	Type II busy period. The server start the new busy period by setting up

Table 4: List of Notations for Appendix